TECHNICAL EVIDENCE REPORT

CoDrone EDU
制御・実験・Git変更
完全詳細報告書

初期 API 実験から多軸ハイブリッド制御、
センサ限界、代替測位、全ログ・全テストまで

対象期間	2026-05-19 – 2026-07-21
作成日	2026-07-27
Repository	PythonProject
HEAD	011b1d17963a70d6f8baa49c19f6f1f870829aa6

Report Scope

Git 履歴	全 25 コミット
PID 飛行ログ	全 42 件
加速度・inertial ログ	全 18 件
球風ログ	全 7 件
Dead reckoning ログ	全 3 件
自動テスト	全 35 件成功
構文検証	全 Python ファイル成功

本書は、確認事実、強い推定、思想上の解釈を分離し、成功例だけでなく、runaway、timeout、欠損ログ、撤回された仮説も同じ重要度で記録する。

EXECUTIVE SECTION

目次

最初に結論 1

第 1 章 0. 本報告書の目的、調査範囲、事実と推定の区別 2

第 2 章 1. 結論の要約 3

第 3 章 2. プロジェクトの最終目標と、その背後にあるモチベーション 5

3.1 2.1 明示された最終目標 5

3.2 2.2 座標ベース制御へ進んだ直接の理由 5

3.3 2.3 なぜ最初に上下方向を試したか 6

第 4 章 3. Git から見た全体像 7

4.1 3.1 現在の Git 状態 7

4.2 3.2 Git 管理方針の変化 7

第 5 章 4. 全コミットの時系列詳細 9

5.1 4.1 2026-05-19 16:41 — `90ddb8b` initial commit 9

5.2 4.2 2026-05-19 17:05 — `1378ff4` チャイム、離陸&着陸 9

5.3 4.3 2026-05-19 17:15 — `c10d4d8` ホバリング 9

5.4 4.4 2026-05-19 18:24 — `4ba456e` 前後左右、z 軸回転、上下 10

5.5 4.5 2026-06-02 16:23 — `09a1ad5` 課題 4.1 まで 10

5.6 4.6 2026-06-02 16:29 — `b60bb33` 課題 4.2 11

5.7 4.7 2026-06-23 16:23 — `9114c3c` PID でいろいろ 11

5.8 4.8 2026-06-23 17:02 — `d697110` 共通制御基盤 12

5.9 4.9 2026-06-23 17:51 — `76274be` 場面別ゲイン調整 12

5.10 4.10 2026-07-07 17:12:07 — `75df208` 実験仕様と記録 12

5.11 4.11 2026-07-07 17:12:13 — `c13d663` 多軸 PID 化 13

5.12 4.12 2026-07-07 17:12:19 — `c144e34` ブレーキ確認 13

5.13 4.13 2026-07-07 17:12:27 — `bf0d1ff` 実機ログと PID プロファイル 13

5.14 4.14 2026-07-07 17:12:36 — 700c91f ワークスペース設定 . . . 14
5.15 4.15 2026-07-07 17:12:58 — b01dfef PID 回帰テスト . . . 14
5.16 4.16 2026-07-07 17:13:36 — a506470 タイムアウトとゲイン調整 . . . 14
5.17 4.17 2026-07-07 17:47 — c001264 追加ログ . . . 14
5.18 4.18 2026-07-07 18:21 — f6232eb 仕様書を書き直し、PID の限界を認識 . . . 14
5.19 4.19 2026-07-07 18:29:58 — 2314db9 PID と回帰テスト更新 . . . 15
5.20 4.20 2026-07-07 18:30:03 — 2f57132 実験記録追記 . . . 16
5.21 4.21 2026-07-07 18:30:09 — a7e88bb 実行ログと解析結果 . . . 16
5.22 4.22 2026-07-07 18:38 — 10fc6f3 Git 漏洩経路遮断 . . . 16
5.23 4.23 2026-07-14 18:26 — 693f019 加速度だけによる位置推定 . . . 16
5.24 4.24 2026-07-14 18:49 — 996a5c3 姿勢制御の堅牢化 . . . 17
5.25 4.25 2026-07-21 17:09 — 011b1d1 カラーセンサ精度実験 . . . 17

第 6 章 5. PID 実験ログから分かったこと 18
6.1 5.1 2026-06-23: 初期 PID の傾向 . . . 18
6.2 5.2 2026-06-30: 基準位置、符号、stale の切り分け . . . 18
6.2.1 163702 . . . 18
6.2.2 164659 . . . 19
6.2.3 165510 / 165807 . . . 19
6.2.4 170026 . . . 19
6.2.5 170651 . . . 19
6.2.6 171035 . . . 20
6.2.7 172047 / 172711 / 174211 . . . 20
6.3 5.3 2026-07-07: 多軸高速化の結果 . . . 20

第 7 章 6. 現在の drone_precision.py の制御方法 22
7.1 6.1 固定符号 . . . 22
7.2 6.2 基準位置取得 . . . 22
7.3 6.3 状態推定 . . . 22
7.4 6.4 PID 本体 . . . 22
7.5 6.5 世界座標と機体座標 . . . 23
7.6 6.6 高速移動の状態機械 . . . 23

7.7 6.7 近傍速度サーボ . . . 23
7.8 6.8 yaw 優先 . . . 23
7.9 6.9 水平ドリフトガード . . . 24
7.10 6.10 高度ガード . . . 24
7.11 6.11 安全終了理由 . . . 24

第 8 章 7. optical flow sensor の評価と、その限界から生まれた新手法 26
8.1 7.1 利用者の結論 . . . 26
8.2 7.2 ログとコードが示す具体的な問題 . . . 26
8.3 7.3 なぜ完全には捨てず、複数案を試したか . . . 26

第 9 章 8. 加速度だけによる位置推定実験 27
9.1 8.1 accel_position_control.py . . . 27
9.2 8.2 実機ログ結果 . . . 27
9.3 8.3 simple_accel_xy.py の未コミット改良 . . . 28
9.4 8.4 2026-07-21 inertial ログ . . . 29

第 10 章 9. 時間較正方式 30

第 11 章 10. コマンド積算 dead reckoning 31
11.1 10.1 運動モデル . . . 31
11.2 10.2 目標制御 . . . 31
11.3 10.3 実機ログ . . . 31

第 12 章 11. カラーセンサ実験 33
12.1 11.1 ハードウェア . . . 33
12.2 11.2 実験方法 . . . 33
12.3 11.3 結果 . . . 33
12.4 11.4 運用上の結論 . . . 33

第 13 章 12. 球を風で 30cm 動かす実験 34
13.1 12.1 目的 . . . 34
13.2 12.2 条件 . . . 34
13.3 12.3 安全上の工夫 . . . 34

13.4 12.4 調整方法 . . . 34
13.5 12.5 現存ログの状態 . . . 35

第 14 章 13. 離陸診断 36

第 15 章 14. テストしたこと 38
15.1 14.1 現在の自動テスト結果 . . . 38
15.2 14.2 test_drone_precision.py . . . 38
15.2.1 停止距離 . . . 38
15.2.2 状態推定 . . . 38
15.2.3 座標変換 . . . 38
15.2.4 固定符号 . . . 38
15.2.5 PID . . . 38
15.2.6 settle 指令 . . . 39
15.2.7 高速状態 . . . 39
15.2.8 ドリフトガード . . . 39
15.3 14.3 test_accel_position_control.py . . . 39
15.4 14.4 test_time_calibrated_xy.py . . . 39
15.5 14.5 テストが保証しないこと . . . 40

第 16 章 15. 実験方法上の工夫 41
16.1 15.1 短距離で符号を確認 . . . 41
16.2 15.2 一次原因を先に直す . . . 41
16.3 15.3 仮説を撤回する . . . 41
16.4 15.4 回数ではなく実時間 . . . 41
16.5 15.5 異常ログを自動学習から除外 . . . 41
16.6 15.6 全周期ログ . . . 41
16.7 15.7 一度に一変数 . . . 42
16.8 15.8 人間の実測を正式なデータとして扱う . . . 42
16.9 15.9 安全処理を通常ロジックと同格に扱う . . . 42

第 17 章 16. ファイル別の役割 43
17.1 16.1 現在の中核 . . . 43

17.2 16.2 optical flow 代替 . . . 43
17.3 16.3 診断・個別実験 . . . 43
17.4 16.4 初期飛行・手動操作 . . . 43
17.5 16.5 非飛行・周辺 API . . . 44
17.6 16.6 文書 . . . 44

第 18 章 17. 当時の思想・考え方の変遷 45
18.1 17.1「動けばよい」から「説明できなければならない」へ . . . 45
18.2 17.2「同じ命令」から「同じ結果」へ . . . 45
18.3 17.3「PID なら解ける」から「観測できるものしか制御できない」へ . . . 45
18.4 17.4「平均性能」から「失敗時の安全」へ . . . 45
18.5 17.5「単一正解」から「複数モデルの使い分け」へ . . . 45
18.6 17.6「最速」の意味の具体化 . . . 45

第 19 章 18. 未解決事項 47
19.1 18.1 optical flow . . . 47
19.2 18.2 カラーセンサ . . . 47
19.3 18.3 上下制御 . . . 47
19.4 18.4 加速度推定 . . . 47
19.5 18.5 時間較正・dead reckoning . . . 47
19.6 18.6 球風実験 . . . 48
19.7 18.7 ソフトウェア . . . 48

第 20 章 19. 現在地点の正確な評価 49
20.1 達成したこと . . . 49
20.2 未達成 . . . 49
20.3 現実的な結論 . . . 49

第 21 章 20. 次に行うべき実験 50
21.1 20.1 センサを同時記録する . . . 50
21.2 20.2 外部測定は評価専用に使う . . . 50
21.3 20.3 上昇・下降を別実験にする . . . 50
21.4 20.4 カラー焼き付きの切り分け . . . 50

21.5 20.5 代替推定の役割分担 . . . 51

第 22 章 21. 最終総括 52

第 23 章 22. 結論・事実・原因・対応策を対応づけた総合判定 53

23.1 22.1 同じプログラムでも飛行結果の再現性が低い . . . 53

23.2 22.2 純粋な PID だけでは最速・精密・堅牢を同時達成しにくい . . . 54

23.3 22.3 optical flow は最終目標に対する単独の信頼源にならない . . . 55

23.4 22.4 加速度二重積分だけでは長時間位置を維持できない . . . 56

23.5 22.5 カラーセンサは通常飛行高度の連続位置観測に適さない . . . 57

23.6 22.6 上昇と下降は対称ではない . . . 58

第 24 章 23. PID 飛行ログ全 42 件の完全一覧 60

24.1 23.1 表の読み方 . . . 60

24.2 23.2 全ログを一件ずつ言語化した評価 . . . 63

24.2.1 20260623_164713 . . . 63

24.2.2 20260623_165545 . . . 63

24.2.3 20260623_170224 . . . 63

24.2.4 20260623_170643 . . . 63

24.2.5 20260623_171101 . . . 63

24.2.6 20260623_171450 . . . 63

24.2.7 20260623_172036 . . . 63

24.2.8 20260623_173526 . . . 63

24.2.9 20260623_173618 . . . 63

24.2.10 20260623_174318 . . . 64

24.2.11 20260630_163702 . . . 64

24.2.12 20260630_164659 . . . 64

24.2.13 20260630_165510 . . . 64

24.2.14 20260630_165714 . . . 64

24.2.15 20260630_165807 . . . 64

24.2.16 20260630_170026 . . . 64

24.2.17 20260630_170651 . . . 64

24.2.18 20260630_171035 . . . 64
24.2.19 20260630_172047 . . . 65
24.2.20 20260630_172711 . . . 65
24.2.21 20260630_174211 . . . 65
24.2.22 20260707_164639 . . . 65
24.2.23 20260707_164950 . . . 65
24.2.24 20260707_165019 . . . 65
24.2.25 20260707_165327 . . . 65
24.2.26 20260707_165345 . . . 65
24.2.27 20260707_165911 . . . 65
24.2.28 20260707_165932 . . . 65
24.2.29 20260707_170225 . . . 66
24.2.30 20260707_170244 . . . 66
24.2.31 20260707_170506 . . . 66
24.2.32 20260707_170526 . . . 66
24.2.33 20260707_170935 . . . 66
24.2.34 20260707_171019 . . . 66
24.2.35 20260707_171632 . . . 66
24.2.36 20260707_171724 . . . 66
24.2.37 20260707_172317 . . . 66
24.2.38 20260707_172418 . . . 66
24.2.39 20260707_173145 . . . 67
24.2.40 20260707_173239 . . . 67
24.2.41 20260707_173920 . . . 67
24.2.42 20260707_173949 . . . 67

第 25 章 24. 全 25 コミットの変更量と、その意味 68

第 26 章 25. 全 35 テストを一件ずつ説明する 70
26.1 25.1 加速度位置制御 12 件 . . . 70
26.2 25.2 drone_precision 17 件 . . . 71
26.3 25.3 時間較正 6 件 . . . 72

26.4 25.4 テスト結果の正しい解釈 . . . 72

第 27 章 26. 用語と数式を省略せず説明する 74

27.1 26.1 PID . . . 74
27.2 26.2 アンチワインドアップ . . . 74
27.3 26.3 stale . . . 74
27.4 26.4 runaway . . . 74
27.5 26.5 ZUPT . . . 74
27.6 26.6 dead reckoning . . . 74
27.7 26.7 世界座標と機体座標 . . . 75
27.8 26.8 停止距離 . . . 75
27.9 26.9 timeout . . . 75
27.10 26.10 settled . . . 75

第 28 章 27. 事実・推定・解釈の証拠台帳 76

28.1 27.1 直接確認できる事実 . . . 76
28.2 27.2 強い推定 . . . 76
28.3 27.3 断定できない解釈 . . . 76

第 29 章 28. 完全版としての最終結論 77

第 30 章 29. PID 以外の現存実験ログを一件ずつ確認した台帳 78

30.1 29.1 球風実験 7 件 . . . 78
30.1.1 ball_wind_20260714_173950.jsonl . . . 78
30.1.2 ball_wind_20260714_174007.jsonl . . . 78
30.1.3 ball_wind_20260714_175404.jsonl . . . 78
30.1.4 ball_wind_20260714_175438.jsonl . . . 78
30.1.5 ball_wind_20260714_175507.jsonl . . . 79
30.1.6 ball_wind_20260714_175824.jsonl . . . 79
30.1.7 ball_wind_20260714_180329.jsonl . . . 79
30.2 29.2 初期加速度位置 CSV 10 件 . . . 79
30.2.1 accel_position_20260714_174347.csv . . . 79
30.2.2 accel_position_20260714_174436.csv . . . 79

30.2.3 accel_position_20260714_174455.csv . . . 80
30.2.4 accel_position_20260714_174527.csv . . . 80
30.2.5 accel_position_20260714_174918.csv . . . 80
30.2.6 accel_position_20260714_175055.csv . . . 80
30.2.7 accel_position_20260714_175224.csv . . . 80
30.2.8 accel_position_20260714_175559.csv . . . 80
30.2.9 accel_position_20260714_175852.csv . . . 80
30.2.10 accel_position_20260714_180130.csv . . . 81
30.3 29.3 改良 inertial XY ログ 8 件 . . . 81
30.3.1 inertial_xy_20260721_162442.csv . . . 81
30.3.2 inertial_xy_20260721_162534.csv . . . 81
30.3.3 inertial_xy_20260721_162638.csv . . . 81
30.3.4 inertial_xy_20260721_162737.csv . . . 81
30.3.5 inertial_xy_20260721_170652.csv . . . 82
30.3.6 inertial_xy_20260721_171307.csv . . . 82
30.3.7 inertial_xy_20260721_171423.csv . . . 82
30.3.8 inertial_xy_20260721_171507.csv . . . 82
30.4 29.4 command dead reckoning ログ 3 件 . . . 82
30.4.1 command_dead_reckoning_20260721_172153.csv . . . 82
30.4.2 command_dead_reckoning_20260721_172837.csv . . . 83
30.4.3 command_dead_reckoning_20260721_172907.csv . . . 83
第 31 章 30. 作業ツリーに残る未コミット状態の意味 84

EXECUTIVE SECTION

最初に結論

本プロジェクトでは、CoDrone EDU を「同じ命令で同じように動かす」段階から、「現在位置・速度・姿勢・センサ鮮度を観測し、風やぶれがあっても指定座標へ戻し、異常時には安全停止する」段階へ発展させた。初期の固定出力・固定時間飛行、単軸 PID、多軸 PID、ログ解析、距離別高速制御を経て、最終的には純 PID だけに頼らず、状態機械、世界座標と機体座標の変換、yaw 優先、最大加速、物理停止距離、最大制動、近傍速度サーボ、水平ドリフト抑制、高度回復、runaway・sensor timeout・attitude fault を組み合わせるハイブリッド制御へ到達した。

一方、実機で「毎回、高速かつ高精度に指定量だけ移動する」という最終目標は未達成である。ソフトウェアの構造と安全性は大きく改善したが、主な制約は制御ゲインではなく位置観測である。optical flow は高い stale 比率、古い値、床・照明・高度・傾斜依存、高速時の破綻を示した。加速度二重積分はゼロ張り付きと大ドリフトの両方を示した。時間較正と command dead reckoning は optical flow に依存しないが、風や物理外乱を直接観測しない。カラーセンサは約 2cm まで近づかなければ判定しにくく、約 3 秒同値を返し続ける焼き付きがある。

したがって、現時点で採るべき方針は、どれか一つのセンサや一つの PID を万能とみなすことではない。optical flow が健全な時は位置観測として使い、短い欠落は IMU と command model で補間し、壁 ToF や既知色マーカーを利用できる時だけ絶対補正し、推定不確実性が限界を越えたら停止する方式である。外部測位を最終制御へ使わないとしても、評価時には外部真値を取り、内部推定が本当に正しいかを検証する必要がある。

Git に残る全 25 コミット、現存する PID ログ全 42 件、加速度・inertial ログ全 18 件、球風ログ全 7 件、command dead reckoning ログ全 3 件、現在の 35 ユニットテストを確認した。自動テスト 35 件はすべて成功し、全 Python ファイルは構文チェックを通過した。ただし、これはコード上の数式・符号・安全条件の成功であって、実機位置精度の完成証明ではない。

以下では、この結論に至った事実、原因、対応策、変更目的、背景、モチベーション、思想の変化、成功、失敗、撤回された仮説、未解決事項を省略せず記述する。

CHAPTER 1

0. 本報告書の目的、調査範囲、事実と推定の区別

本報告書は、このリポジトリでこれまで行われた CoDrone EDU に関する学習、基礎機能確認、飛行実験、PID 制御、ログ解析、センサ評価、代替測位手法、安全対策、テスト、文書化、および Git 上の変更を、可能な限り漏れなく一つに統合したものである。

調査対象は次のとおりである。

- Git に残る全 25 コミット
- 各コミットのメッセージ、変更ファイル、追加・削除行、主要差分
- 現在の追跡対象ファイル
- 現在の未コミット変更および未追跡ファイル
- `flight_logs` に現存する PID 飛行ログ
- `accel_position_logs` の加速度積分実験ログ
- `ball_wind_logs` の球風移動実験ログ
- `command_dead_reckoning` の CSV ログ
- `experiment_logs` と `memo` の仕様、解析、実験記録
- 現在のユニットテストと全 Python ファイルの構文検証
- 利用者から今回明示された、カラーセンサ、飛行再現性、上下非対称性、optical flow sensor への評価

記述は以下の 3 種類に分ける。

1. **確認事実**コード、ログ、コミット、文書、または今回の利用者証言に直接書かれている内容。
2. **強い推定**変更の直前の失敗ログと直後のコード変更が明確に対応しており、当時の意図をかなり高い確度で復元できる内容。
3. **解釈・思想の推定** Git は「何を変えたか」は示すが、人間の内面を完全には記録しない。そのため、動機や思想は根拠を併記し、断定しない。

この区別は重要である。たとえば「optical flow sensor はカス」という評価は利用者の明示的な実機評価としてそのまま重く扱える。一方、「なぜその日にその実験を選んだか」の細部は、変更順序、ログ、文書から推定するしかない。

CHAPTER 2

1. 結論の要約

このプロジェクトの本質は、単なるドローン API の練習から始まり、次第に次の問題へ収束していった過程である。

> 同じ命令を出しても毎回同じ場所へ行かず、風、機体差、姿勢、*yaw*、センサ遅延、*optical flow* の破綻によって結果が変わる小型ドローンを、内蔵センサだけで、指定座標へ、できるだけ速く、精密に、安全に移動させられるか。

開発思想は大きく 5 段階に変化した。

1. **API と機体特性を一つずつ触って理解する段階**離陸、着陸、ホバリング、LED、ブザー、前後左右、上下、yaw、各種センサを小さな独立スクリプトで試した。
2. **時間・固定出力による開ループ制御の段階**一定出力を一定時間与え、図形飛行や手動操作を行った。しかし、再現性と外乱耐性が低い。
3. **位置センサを使う閉ループ PID の段階**位置誤差を pitch/roll/throttle へ戻し、指定距離へ収束させようとした。ログ、自動調整、整定判定、安全停止も追加した。
4. **純 PID の限界を認識し、ハイブリッド制御へ移る段階**世界座標と機体座標の変換、yaw 優先、速度サーボ、制動距離、bang-bang 型の最大加速・最大制動、stale 時間判定、runaway、高度回復、ドリフト抑制を組み合わせた。
5. **optical flow 依存を外す複数の代替案を並行検討する段階**加速度二重積分、姿勢角由来の加速度、ZUPT、時間較正、コマンド積算型 dead reckoning、壁 ToF 補正、低高度カラー・マーカー補正を試した。

最終的に得られた最も重要な知見は次のとおりである。

- 飛行中のぶれが大きく、同じプログラムでも再現性が低い。
- 風や物理的外乱があるため、時間だけで動かす開ループ制御では目標位置を保証できない。
- optical flow 由来位置は更新停止、stale、高速移動中の破綻、姿勢・床・照明依存が大きく、主たる弱点になった。
- 位置センサが観測を失った後、内蔵センサだけで真の絶対位置を厳密に復元することはできない。
- 上昇と下降の応答は対称ではない。上昇は得意だが下降は苦手で、同じ発想・同じゲインを上下に対称適用すべきではない。
- yaw がずれると、世界座標の x/y 誤差と機体の pitch/roll 方向がずれ、円運動や発散が起こる。
- センサ更新停止は「制御ループ何回分止まったか」ではなく「実時間で何秒止まったか」で扱う方が適切である。
- 最速性を求める場合、純 PID だけで全距離を走るより、遠方では最大加速、停止距離内では最大制動、近傍では速度サーボという状態機械が合理的である。
- カラーセンサは約 2cm 程度の低高度でなければ色判定が難しく、実運用では着地後判定に近い。
- カラーセンサは約 3 秒同じ値を返し続ける焼き付き・フリーズを頻発した。取得周期を sleep で落としても体

感頻度が変わらず、単純なポーリング過多説は支持されなかった。

- 加速度二重積分は、量子化、バイアス、ノイズ、符号、姿勢変化に極端に弱い。ログでは推定値がゼロに張り付く場合と大きくドリフトする場合の両方が確認された。
- コマンド積算型 dead reckoning は自己整合的な推定値を作れるが、外乱を直接観測しない。そのため、統計的停止判定が安定していても真値保証にはならない。
- 自動テストは現在 35 件すべて成功しているが、テストが保証するのは数式、符号、状態遷移などのソフトウェア上の不変条件であり、実機の位置精度ではない。

CHAPTER 3

2. プロジェクトの最終目標と、その背後にあるモチベーション

3.1 2.1 明示された最終目標

仕様書群から読み取れる目標は明確である。

- x, y, z, yaw の相対指令を与える。
- 指定量だけ移動し、目標位置で停止する。
- 風、電波干渉、センサ誤差、軽い物理的妨害があっても再収束する。
- 可能な範囲で最高速に加速し、目標直前で急制動する。
- 最終位置では低速で精密に合わせる。
- センサ異常、低高度、姿勢崩れ、発散時には安全停止する。
- 変更理由と飛行結果をログとテストから追跡できるようにする。

優先順位は、文書上では次の順で整理されている。

1. 最速性
2. 精密性
3. 堅牢性
4. 保守性

ただし、実装上は安全がこれらの前提になっている。最大出力を使う設計と、runaway・高度・姿勢・センサ timeout のガードが同時に実装されているのは、「速さを求めるからこそ、異常時に即座に止める必要がある」という思想の表れである。

3.2 2.2 座標ベース制御へ進んだ直接の理由

利用者の明示説明では、飛行中にあらゆる方向へのぶれが激しく、同じプログラムでも結果の再現性が低かった。扇風機の風が進行方向にある、途中で風を受ける、といった外的要因があっても、行きたい方向へ直進させたいというのが PID と座標ベース制御の直接のモチベーションである。

これは時間制御との決定的な違いである。

- 時間制御:「この出力を 1 秒出せば、だいたいこの距離進む」という前提。
- 座標閉ループ制御:「現在どこにいるかを読み、目標との差が残る限り補正する」という前提。

前者は無風・同一バッテリー・同一床・同一姿勢に依存する。後者は観測が正しい限り外乱後も戻れる。しかし、こ

のプロジェクトでは後に「観測そのものが信用できない」という第二の問題へ到達した。

3.3 2.3 なぜ最初に上下方向を試したか

利用者証言では、上下運動が見た目で最も分かりやすいため、最初の PID 実験対象に選ばれた。その結果、上昇と下降のしやすさが大きく異なることが分かった。

- 上昇: 推力を増せば比較的素直に反応する。
- 下降: 推力を落とすだけでは応答が遅い、機体が不安定になりやすい、地面付近の気流や高度センサ遅延の影響を受ける可能性がある。

この非対称性は、後の `throttle` 制御における近傍ゲイン弱化、速度ブレーキ、高度回復ガード、Z 軸専用の速度上限へつながっていると解釈できる。

CHAPTER 4

3. Git から見た全体像

4.1 3.1 現在の Git 状態

報告書作成時点で以下の状態だった。

- ブランチ: `master`
- `origin/master` より 1 コミット先行
- ローカル HEAD: `011b1d1`
- リモート HEAD: `996a5c3`
- 追跡済み変更:
  - `experiment_logs/README.md`
  - `simple_accel_xy.py`
- 未追跡の主要コード:
  - `accel_position_control.py`
  - `ball_wind_experiment.py`
  - `codrone_takeoff_diagnose.py`
  - `command_dead_reckoning.py`
  - `time_calibrated_xy.py`
  - `test_accel_position_control.py`
  - `test_time_calibrated_xy.py`
- 未追跡の主要ログ:
  - `accel_position_logs/`
  - `ball_wind_logs/`
  - `experiment_logs/2026-07-14_球風移動実験.md`

つまり、Git の最新コミットはカラーセンサ実験だが、作業ツリーにはその先の大規模な代替測位・球風・離陸診断・時間較正実験がある。現在の研究状態は「コミット済みの完成状態」ではなく、複数案を比較中の研究ノートに近い。

4.2 3.2 Git 管理方針の変化

初期には `.idea` と実機ログが Git へ入っていた。2026-07-07 の `10fc6f3` で、次がまとめて Git 管理から外された。

- IDE 固有状態 `.idea/`
- Python キャッシュ
- 仮想環境
- 秘密情報や鍵になり得るファイル

- ローカル実行ログ
- `flight_logs/`

コミットメッセージの「漏洩経路を遮断」は、単なる整理ではなく、個人環境、秘密情報、巨大な実行生成物を誤って公開しないための防御である。同時に、約 17,916 行相当のログが追跡対象から削除された。ログ自体は作業ディレクトリに残っており、解析には使えるが、Git 履歴の軽量化と情報管理を優先した判断である。

CHAPTER 5

4. 全コミットの時系列詳細

5.1 4.1 2026-05-19 16:41 — `90ddb8b` initial commit

PyCharm の初期テンプレートと IDE 設定を登録した。まだドローン制御はない。

読み取れる意味:

- 研究・課題のための Python プロジェクトを作った出発点。
- この時点では制御思想は未形成。
- `.idea` まで登録しており、Git 運用は学習初期の状態だった。

5.2 4.2 2026-05-19 17:05 — `1378ff4` チャイム、離陸＆着陸

最初の実機接続段階である。

- `Drone()` 生成
- `pair()`
- 離陸
- 着陸
- LED
- Westminster Chimes のブザー演奏
- `close()`

モチベーションの推定:

- まず通信、アクチュエータ、飛行 API が動くことを小さく確認する。
- 音と LED は飛行制御には直接不要だが、機体との通信成功を人間が即座に確認できる。
- 初期段階では「精密制御」より「API を一通り動かす」ことが主目的。

5.3 4.3 2026-05-19 17:15 — `c10d4d8` ホバリング

離陸と着陸の間にホバリングを追加した。

意味:

- 離陸直後にすぐ次動作へ進むのではなく、安定時間を置く発想が現れた。
- 後の基準位置取得、センサ校正、post-takeoff wait の原型。

5.4 4.4 2026-05-19 18:24 — `4ba456e` 前後左右、z 軸回転、上下

基本 6 自由度の操作確認へ進んだ。

- pitch による前後
- roll による左右
- yaw による回転
- throttle による上下
- `xy-axis.py`
- `z-axis.py`

当時の方式は固定出力・固定時間の完全な開ループ制御だった。`xy-axis.py` では roll 30 を 4 秒、pitch 30 を 4 秒など、順番に四方向へ動かす。`z-axis.py` では強い上下指令を繰り返す意図が見えるが、`for i in 10` は Python として実行時エラーになるため、実験用の粗い試作だったと考えられる。

5.5 4.5 2026-06-02 16:23 — `09a1ad5` 課題 4.1 まで

多数の小スクリプトが一括追加された。これはセンサと周辺機能を一つずつ分離して試すフェーズである。

- `30sec.py`: 30 秒飛行。`move()` 処理時間、時刻取得、Python ループによる時間誤差への注意。
- `3_1_sine.py`: throttle を交互に切り替える上下波形的な飛行。
- `Gyro_sensor.py`: x/y/z 角度の連続読取り。
- `barometer.py`: 気圧の連続読取りと高度別比較案。
- `battery.py`: 飛行前後のバッテリー比較。
- `buzzer.py`: ブザー。
- `doremifasollasido.py`: 音階。
- `flight_time.py`: 離陸から着陸までの経過時間。
- `front_range.py`: 前方 ToF。コメント上の実測範囲は約 3〜210cm。
- `function.py`: 正方形飛行を関数化。
- `height.py`: 高度・下方距離。コメント上の限界は約 150cm。
- `pentagon.py`: pitch と yaw による五角形飛行。
- `practice_random.py`: ランダム LED。
- `random_noise.py`: ランダム音。
- `sensor_1.py`: 複合センサ確認の入口。
- `shine.py`: LED グラデーション。
- `temperature.py`: 温度連続取得。
- `time-based_flight.py`: 指定秒数だけ `move()` する関数。
- `user_input.py`: 入力回数に応じた回転。

このコミットの工夫:

- 一つのスクリプトに全機能を詰めず、機能ごとに分離した。

- 後の不具合切り分けに使える最小再現コード群になっている。
- 前方距離と高さについて実測限界をコメントに残した。
- 時間制御の誤差要因を早い段階で認識していた。

5.6 4.6 2026-06-02 16:29 — b60bb33 課題 4.2

`user_input.py` を実装し、ユーザ指定回数だけ yaw 回転する操作を追加した。

ここではまだ「回転数」を実測角度で閉ループ制御せず、固定時間の yaw 操作を回数分行う。ユーザ指令を制御パラメータへ変換する最初の段階である。

5.7 4.7 2026-06-23 16:23 — 9114c3c PID でいろいろ

プロジェクトの転換点である。

追加・変更:

- `z-pid.py`: 高度 80cm を P/I/D で維持
- `move.py`: x/y 位置を PID 制御し、PC キーボードで目標座標を変更
- `keyboard_input.py`: 8 方向＋上下＋ yaw の手動操作
- `main.py`, `pid_test1.py`: 前進距離を PID で制御する大きな実装
- `brake.py`: 当初空ファイル

初期 PID の工夫:

- `dt` を実測。
- 積分値を制限してワインドアップを防止。
- 出力を-100〜100 に制限。
- タイムアウト。
- 目標誤差内に一定時間留まるまで完了としない整定判定。
- 例外や Ctrl+C 後にも着陸する `finally`。
- バッテリー残量による離陸拒否。
- x だけでなく y/z もログ表示して副作用を観察。

当時の思想:

- 「一定時間動かす」のではなく、「誤差を見て戻す」。
- 軽量機はオーバーシュートしやすいため、積分制限と出力制限が必須。
- 一瞬許容範囲へ入っただけでは到達扱いにしない。
- 手動操作と PID 操作を併置し、符号と機体応答を現場で確認する。

5.8 4.8 2026-06-23 17:02 — d697110 共通制御基盤

main.py と pid_test1.py に重複していた制御を drone_precision.py へ集約した。

追加された主要概念:

- 共通 PID 制御器
- 逐次 JSONL ログ
- テキストログ
- 1 飛行の summary
- 過去ログの集計
- 前回結果に基づく軽い自動チューニング
- 距離別プロファイル
- CLI 入口の一本化

モチベーション:

- 同じロジックが複数ファイルに分岐すると修正漏れが起きる。
- 実機は再現性が低いため、「見た目」だけでなく全周期の数値を残す必要がある。
- 一回の結果に過剰反応せず、過去結果と比較したい。
- ゲイン調整をコード編集ではなくデータとして管理したい。

5.9 4.9 2026-06-23 17:51 — 76274be 場面別ゲイン調整

roll、throttle、pitch を一律に扱わず、状況別・軸別に調整する設計へ進んだ。

強い推定:

- 上下・前後・左右で応答が異なるため、単一 PID パラメータでは不十分だった。
- 距離、目標近傍、速度、出力飽和に応じたゲイン・上限変更が必要になった。
- 利用者が述べた「上昇は得意、下降は苦手」という非対称性とも一致する。

5.10 4.10 2026-07-07 17:12:07 — 75df208 実験仕様と記録

実験記録を Markdown で体系化した。

- 2026-06-30 変更詳細
- 2026-07-07 多軸最速 PID
- 実験履歴サマリー
- 解析方法
- experiment_logs/README.md
- 技術仕様

重要な思想:

- 実験はコードだけでは再現できない。条件、期待、結果、解釈、次回変更を残す。
- 変更→実機飛行→ログ要約→次回仮説という循環を明文化。
- 評価軸を x/y/z/yaw/時間/オーバーシュート/センサ鮮度に固定。
- 成功例だけでなく誤判断と撤回も記録。

5.11 4.11 2026-07-07 17:12:13 — c13d663 多軸 PID 化

drone_precision.py は約 1,358 行追加、552 行削除という大規模再設計を受けた。

主な変更:

- x/y/z/yaw を同時に制御。
- 軸ごとの PID。
- 距離別最大出力。
- stale 判定。
- 高速移動フェーズ。
- 全軸ログ。
- CLI で複数軸を指定。
- 終了コードを外部へ返す。

モチベーション:

- x だけ合っても、y へ流れ、高度がずれ、yaw が回れば総合的には失敗。
- 直列に一軸ずつ直すより、全軸を毎周期監視・補正したい。
- 「最速」を目指すなら、補助軸の崩れで主軸を何度もやり直す時間を減らす必要がある。

5.12 4.12 2026-07-07 17:12:19 — c144e34 ブレーキ確認

brake.py へ pair/land/close の短い確認コードを追加した。

ただし実装は別々の Drone() インスタンスに対して pair()、land()、close() しており、通常意図する同一接続の安全停止確認としては不自然である。実験的な緊急着陸確認コード、または急いで作った最小スクリプトと考えられる。

5.13 4.13 2026-07-07 17:12:27 — bf0d1ff 実機ログと PID プロファイル

2026-06-23、06-30、07-07 の大量の飛行ログ、距離別 PID プロファイル、最新ログ、履歴解析を Git へ登録した。

意味:

- 実機結果をコードと同じ重要度の成果物として扱った。

- 100mm、300mm、600mm、2000mm で別プロファイルを持たせた。
- 距離によって必要な加速・制動・精密停止が違うという認識。

5.14 4.14 2026-07-07 17:12:36 — `700c91f` ワークスペース設定

IDE 設定更新。制御そのものではないが、当時は`.idea` を共有対象にしていた。

5.15 4.15 2026-07-07 17:12:58 — `b01dfef` PID 回帰テスト

278 行の `test_drone_precision.py` を追加した。

初期のテスト思想:

- 実機を毎回飛ばさなくても、停止距離、速度計算、PID 飽和、積分、状態遷移を検証。
- 実機ログで見つけたバグを、同じ形で再発しないようテストへ固定。
- ハードウェア依存部を fake drone で置き換える。

5.16 4.16 2026-07-07 17:13:36 — `a506470` タイムアウトとゲイン調整

実機実験の安全性と収束性に合わせてタイムアウト・制御ゲインを変更した。

この種の変更から読み取れる思想:

- 理論値より実機ログを優先する。
- 早すぎる停止は到達途中を失敗にする一方、長すぎる timeout は暴走リスクを増やす。
- ゲインだけでなく、観測更新周期と試験距離を合わせる必要がある。

5.17 4.17 2026-07-07 17:47 — `c001264` 追加ログ

7 回分の実機ログを追加した。ログでは runaway 終了が多数確認される。

重要な点:

- 失敗を削除せず保存した。
- 「制御が動いた」ことではなく、どの終了理由でどの誤差が残ったかを評価した。
- 短時間で複数条件を試し、runaway ガードが実際に作動することを確認した。

5.18 4.18 2026-07-07 18:21 — `f6232eb` 仕様書を書き直し、PID の限界を認識

コミットメッセージ自体が思想転換を明示する。

追加文書:

- 要件定義
- 制御仕様
- 実装手順

結論:

- 純粋な PID は最終方式ではない。
- 状態機械
- センサ信頼度判定
- yaw 付き世界座標→機体座標変換
- 距離別 bang-bang 加速・制動
- 残距離ベース速度サーボ
- 水平ドリフト抑制
- 高度回復
- 低速最終補正

思想的には、「PID 係数をさらに追い込めばすべて解決する」という前提を捨て、「観測、座標系、物理的停止距離、フェーズ、異常状態を明示的にモデル化する」方向へ進んだ。

5.19 4.19 2026-07-07 18:29:58 — `2314db9` PID と回帰テスト更新

制御本体 339 行追加、38 行削除、テスト 160 行追加。

主要な改善:

- 世界座標誤差を yaw で機体座標誤差へ変換。
- body 速度も同様に変換。
- yaw 誤差が大きいと水平出力を縮小。
- yaw がさらに大きければ水平出力を 8 へ制限。
- 水平誤差増幅・振動を検出して出力を 0.55 または 0.35 倍へ。
- pitch/roll 積分を 0.45 倍へ減衰。
- 近傍速度サーボ。
- 停止距離に入ったら PID 出力よりブレーキを優先。
- 水平移動による傾斜を見込んだ throttle 補償。
- 高度急落時は水平・yaw を捨て、throttle 80 で回復。
- `sensor_timeout`, `altitude_fault`, `attitude_fault`。
- センサ timeout を 1.20 秒から 0.65 秒へ厳格化。

追加テスト:

- 座標変換の可逆性。
- yaw 符号。
- 水平ドリフトガード。

- 遠く低速なら目標方向へ駆動。
- 近く高速なら逆向きに制動。
- 早すぎる逆指令を目標方向へ戻す。

5.20 4.20 2026-07-07 18:30:03 — `2f57132` 実験記録追記

コード変更とログ解析結果を実験記録へ反映した。研究の知識をコードだけに閉じ込めないためのコミットである。

5.21 4.21 2026-07-07 18:30:09 — `a7e88bb` 実行ログと解析結果

履歴解析と最新実行ログを更新。変更直後の実機結果を保存した。

5.22 4.22 2026-07-07 18:38 — `10fc6f3` Git 漏洩経路遮断

`.gitignore` を追加し、IDE、仮想環境、秘密情報、証明書、ログ、DB、`flight_logs` を除外した。

思想:

- 実験速度だけでなくリポジトリ衛生と安全を重視。
- ローカル生成物とソースコードを分離。
- 公開リポジトリに個人環境や秘密情報を載せない。

5.23 4.23 2026-07-14 18:26 — `693f019` 加速度だけによる位置推定

`simple_accel_xy.py` を追加。optical flow 位置 API に依存しない試みである。

基本方式:

- 離陸前後の加速度バイアス校正。
- raw 加速度を $\mathrm{cm/s^2}$ へ換算。
- deadband。
- low-pass filter。
- yaw で機体座標加速度を世界座標へ回転。
- 加速度を速度へ積分。
- 速度を位置へ積分。
- 速度リークでドリフトを抑制。
- ZUPT で静止時速度をゼロへ。
- 位置誤差と速度から pitch/roll を生成。
- CSV へ raw 値、推定値、指令を保存。

モチベーション:

- optical flow が信用できないなら、IMU だけで相対位置を作れないか。

- 外部測位を使わず、内蔵センサのみという制約を守る。
- 生加速度から位置までの推定過程をすべてログ化して、どこで破綻するかを見たい。

5.24 4.24 2026-07-14 18:49 — `996a5c3` 姿勢制御の堅牢化

`simple_accel_xy.py` を大幅更新した。

主な改善:

- yaw 保持。
- 世界座標と機体座標の変換。
- ZUPT 条件の追加。
- 加速度 deadband と LPF。
- 速度リーク。
- 出力・許容誤差・符号を設定化。
- CSV ログ拡充。
- 例外時もゼロ指令、着陸、close。

これは「加速度積分だけ」では足りず、姿勢が変わると加速度の軸も回転するため、yaw と機体座標を同時に扱う必要があると分かった結果である。

5.25 4.25 2026-07-21 17:09 — `011b1d1` カラーセンサ精度実験

`sensor_1.py` をカラーセンサ連続取得へ変更し、0.1 秒周期で 2 つの RGB センサ値を表示した。

コミットメッセージに実験結論が詳細に保存されている。

- 色判定可能な高さは約 2cm 程度。
- 色タスクでは実質的に着地後判定になる可能性が高い。
- 約 3 秒間、同じ値を返し続けるフリーズ・焼き付きが頻発。
- 本来はシームレスに値が更新されるべき。
- 取得速度が原因という仮説を立て、sleep で周期を落とした。
- 体感では発生頻度が変わらず、取得速度だけでは説明できなかった。
- 原因は未解決。

このコミットは、短いコード変更以上に、ハードウェア制約と否定された仮説をコミットメッセージへ残した点に価値がある。

CHAPTER 6

5. PID 実験ログから分かったこと

6.1 5.1 2026-06-23: 初期 PID の傾向

主なログ:

実行	時間 [s]	最終 x 誤差 [mm]	y 誤差 [mm]	z 誤差 [mm]	特徴
164713	19.98	72.3	27.8	22.3	timeout 近くまで使用
170224	19.95	81.3	-99.6	26.3	y が悪化
170643	19.99	-13.8	-3274.7	10.6	y が壊滅的に発散
171101	20.00	87.0	-1008.8	13.2	y が大きく発散
171450	14.96	50.1	106.3	-17.7	やや短時間
172036	19.95	31.7	89.4	-12.4	x 改善、y 残差
173526	19.99	-86.7	3.1	-20.7	x オーバーシュート
173618	19.96	26.6	-41.7	-26.3	比較的均衡
174318	13.29	-3.9	-18.0	-2.9	初期群の最良

結論:

- x は数 mm まで合う回がある。
- しかし同じ系統の制御でも結果が大きくばらつく。
- y は最も不安定で、数 m 級に壊れる。
- z は y より良いが数 cm 残る。
- 多くが約 20 秒を使い、最速目標には遠い。
- 単発の最良値だけで成功と判断してはいけない。

6.2 5.2 2026-06-30: 基準位置、符号、stale の切り分け

6.2.1 163702

- 時間: 1.88 秒
- x 誤差: 1967.1mm
- z 誤差: -759.7mm
- stale 比率: 0.6774

原因:

- 古い位置を開始基準に採用。
- 高度 0mm を基準と誤認。
- 実高度約 800mm が入ると強い下降指令。
- stale 停止が早すぎた。

対策:

- 離陸後の新鮮な複数サンプルから基準を取得。

6.2.2 164659

- x 誤差: 2424.4mm
- stale 比率: 0.7024

基準位置は改善したが、x が目標と逆へ進んだ。ここで pitch 符号疑いが強まった。

6.2.3 165510 / 165807

- 300mm 目標に対し、x 誤差 1370.1mm / 659.4mm
- どちらも負方向へ進行
- stale 比率 0.76 / 0.72

一度は `pitch_sign=-1` が誤りと判断された。

6.2.4 170026

- 100mm 目標
- x 誤差 45.5mm
- y 誤差 1.2mm
- stale 比率 0.8056

この回だけを見ると目標方向へ進んだため、「符号より stale が主因」という仮説へ一時的に変更した。

ここで重要なのは、当時の判断が後に撤回されたことである。実験記録はその撤回を隠していない。

6.2.5 170651

- x 誤差 2415.3mm
- stale 比率 0.037
- x 速度ピーク 951.5mm/s

位置更新は十分あるのに 2m 以上逆走した。このログにより「stale が主因」という直前の仮説は否定され、`pitch_sign=-1` の誤りが確定的になった。

対策:

- 符号を学習プロファイルから外して固定。
- 短距離初動の出力を抑制。
- 連続逆走で runaway 停止。

6.2.6 171035

- x 誤差 64.2mm
- y 誤差-44.6mm
- z 誤差 26.6mm
- z 速度ピーク 79.0mm/s

逆走は止まったが上下振動が目立った。

原因仮説:

- `throttle.min_output=4` が目標近傍でも上下指令を強制。
- 自動調整で kp/ki が増えすぎた。
- 高度センサ遅延に対し制御が攻めすぎ。

対策:

- throttle 最小出力を 0 へ。
- 60mm 以内でゲイン低下。
- 45mm、25mm で出力上限段階低下。
- 速度ブレーキ。
- 20mm 以内で積分減衰。
- 12mm 以内かつ低速で throttle 0。
- 高速上下振動時は kp/ki を下げ、kv/kd を上げる自動調整へ。

6.2.7 172047 / 172711 / 174211

- 172047: x 誤差 27.1mm、z 誤差 51.9mm
- 172711: x 誤差 1065.9mm、stale 0.73
- 174211: 18.13 秒で runaway、x 誤差 72.8mm

この時期は、stale 中の速度保持、停止距離予測、高速区間と精密区間で異なる runaway 閾値、異常終了を自動学習へ使わない処理へ発展した。

6.3 5.3 2026-07-07: 多軸高速化の結果

初期の 11 回程度は 0.66〜4.27 秒で runaway 終了した。安全ガードは機能したが、制御性能としては不合格である。

後半は timeout まで飛行するようになった。

実行	終了	時間 [s]	x 誤差	y 誤差	z 誤差
170935	timeout	19.95	-33.6	61.0	62.8
171019	timeout	19.97	-16.7	47.2	46.4
171632	timeout	34.95	-57.5	-60.7	62.1
171724	timeout	34.99	66.7	-71.2	54.0
172317	timeout	34.94	43.4	-76.0	798.4
172418	timeout	34.93	-4.7	69.5	30.0
173145	timeout	34.94	18.3	-81.4	-20.4
173239	timeout	34.97	127.9	-1.2	7.5
173920	attitude_fault	1.08	191.2	98.2	35.1
173949	timeout	34.93	109.0	-82.6	-3.1

読み取れること:

- runaway 即停止から、長時間制御継続へは改善した。
- ただし「長く飛べる」と「収束する」は別。
- x だけ良い回でも y または z が残る。
- 172317 では z が約 798mm 崩れ、高度安全性が重大。
- 173920 では yaw/姿勢異常を検出して 1.08 秒で停止。
- 最速目標に対し 35 秒 timeout は長すぎる。
- これが純 PID の限界認識と状態機械化を裏付ける。

CHAPTER 7

6. 現在の drone_precision.py の制御方法

7.1 6.1 固定符号

実機結果から、次を安全・機体固有の固定値として扱う。

```
pitch = +1
roll = -1
throttle = +1
yaw = -1
```

符号を自動チューニング対象にしない理由は、誤学習した符号が 2m 級の逆走を起こしたためである。

7.2 6.2 基準位置取得

離陸直後の単一サンプルを使わず、複数の新鮮なサンプルを取得する。高度が十分あるか、位置が実際に変化したかも見る。

目的:

- 離陸前の 0 値や古いキャッシュを基準にしない。
- 高度 0mm 誤認による急下降を防ぐ。

7.3 6.3 状態推定

- 位置パケットが新鮮なときだけ速度更新。
- 速度計算の `dt` は制御周期ではなく、新鮮な位置パケット間隔。
- stale 中は最後の有効速度を保持または減衰。
- `stale_cycles` だけでなく `stale_duration_sec` を記録。

この変更は、位置センサ更新周期が制御ループ周期より遅い状況で、同じ値が返るたびに「異常回数」を増やす誤判定を防ぐ。

7.4 6.4 PID 本体

- P: 現在誤差
- I: 持続外乱や定常偏差
- D: 誤差変化
- kv: 観測速度への減衰
- 出力飽和中は積分を増やさないアンチワインドアップ
- 近傍では minimum output を無効化

- stale や大姿勢誤差時は積分停止・減衰

7.5 6.5 世界座標と機体座標

位置センサの x/y は離陸時基準の世界座標、pitch/roll は現在向いている機体座標である。yaw が 0 でなければそのまま対応しない。

変換:

```
body_x = world_x cos(yaw) + world_y sin(yaw)
body_y = -world_x sin(yaw) + world_y cos(yaw)
```

これを誤差と速度の両方へ適用する。

7.6 6.6 高速移動の状態機械

各軸は概ね次の状態を持つ。

1. `acquire`
2. `accelerate`
3. `brake`
4. `settle`

x/y は 500mm 以上、z は 300mm 以上で高速フェーズを許可する。短距離で最大加速すると、センサ遅延中に停止距離を使い切るためである。

停止距離:

```
v² / (2a) + v × sensor_latency + margin
```

stale 中は、最後の有効速度と stale 経過時間から残距離を予測する。

7.7 6.7 近傍速度サーボ

残距離から、今から止まれる速度を計算する。

```
reachable_speed = sqrt(2 × brake_accel × remaining_distance)
desired_speed = min(max_speed, reachable_speed × 係数)
```

その目標速度と現在速度の差、および位置誤差から指令を作る。目標近くで速すぎれば、PID がまだ前進を要求していてもブレーキを優先する。

7.8 6.8 yaw 優先

- yaw 誤差 8 度超: 水平出力を段階的に縮小。
- 22 度超: pitch/roll を最大 8 へ。

- 55 度超: attitude_fault。

目的:

- 向きがずれたまま強く前進すると、世界座標上では斜めまたは円運動になる。
- まず方位を戻し、その後に水平移動する。

7.9 6.9 水平ドリフトガード

直近履歴から以下を検出する。

- 最近の最良誤差より大幅に悪化。
- x/y の符号反転が多い。
- 直前区間より平均誤差が増加。

対策:

- pitch/roll を 0.55 または 0.35 倍。
- pitch/roll 積分を 0.45 倍。

7.10 6.10 高度ガード

目標高度が十分高いのに急落した場合:

```
pitch = 0
roll = 0
yaw = 0
throttle = 80
```

水平移動を捨てて高度回復を優先する。これは、傾斜中に下方 optical flow/ToF がさらに壊れ、床へ近づく悪循環を断つためである。

7.11 6.11 安全終了理由

- settled: 誤差と速度が規定時間内に収まった。
- timeout: 時間切れ。
- runaway: 最良到達点から連続して離脱。
- sensor_timeout: 位置更新が 0.65 秒以上停止。
- altitude_fault: 危険高度。
- attitude_fault: yaw/姿勢の大崩れ。
- 例外・ユーザ中断。

異常終了ログは自動チューニングへ使わない。壊れた飛行を「性能データ」として学習すると、誤ったゲインや符号

を強化するためである。

CHAPTER 8

7. optical flow sensor の評価と、その限界から生まれた新手法

8.1　7.1 利用者の結論

利用者の表現は「optical flow sensor はカス」である。技術的に言い換えると、このプロジェクトが求める「外乱下でも厳密・高速・再現可能な相対位置制御」の主要観測器としては、信頼性が不足しているという評価である。

8.2　7.2 ログとコードが示す具体的な問題

- 同じ位置が長く返る。
- stale 比率が 70〜80% に達する回。
- 高速移動・傾斜時に更新が粗い。
- 古い値が離陸後の基準として採用される。
- 更新が戻った瞬間に大きな速度として計算される。
- 床模様、照明、高度、傾きに依存。
- yaw がずれると座標と指令方向が一致しない。
- センサが滑った後、真の絶対位置を復元できない。

8.3　7.3 なぜ完全には捨てず、複数案を試したか

optical flow は問題が多いが、加速度だけの位置推定も累積誤差が大きい。そこで、単一センサに賭けるのではなく、次の代替・補完手法が作られた。

1. IMU 加速度二重積分
2. 姿勢角から重力成分相当の水平加速度を推定
3. IMU と姿勢由来加速度の blend
4. ZUPT
5. 時間較正
6. コマンド積算 dead reckoning
7. 壁 ToF による絶対補正
8. 低高度カラー・マーカーによる位置スナップ

これは「センサ A が悪いからセンサ B へ単純置換」ではなく、各観測の弱点を別の情報で部分補正する思想である。

CHAPTER 9

8. 加速度だけによる位置推定実験

9.1 8.1 accel_position_control.py

より構造化された加速度専用制御器である。

推定処理:

- 1 raw 単位を 100mm/s² として換算。
- 静止平均で x/y/z バイアスを求める。
- LPF 係数 0.35。
- deadband 140mm/s²。
- dt 最大 0.20 秒。
- 速度リーク 0.12/s。
- 台形積分で位置。

制御:

- kp=0.095 command/mm
- kv=0.018 command/(mm/s)
- pitch/roll 上限 35
- 遠方の最小移動出力 24
- 位置許容 8cm
- 速度許容 22cm/s
- 0.55 秒連続で両条件を満たすと settled

工夫:

- センサ読取りを Protocol で抽象化し、fake reader でテスト。
- 加速度符号と指令符号を別設定。
- 距離に応じて timeout を自動延長。
- 離陸成功を高度だけで判定しない。
- flight state と movement state が飛行中なら、高度 0 でも警告付きで続行。
- state/sensor error bitmask をデコード。
- CSV と summary JSON を分離。

9.2 8.2 実機ログ結果

実行	目標 x[cm]	推定 x[cm]	最終 x 誤差 [cm]	終了
174347	30	-15.68	45.68	timeout
174527	60	0.00	60.00	timeout
174918	30	0.00	30.00	timeout
175852	30	-150.29	180.29	timeout

重要な観測:

- raw x/y が 0 に量子化され続けると、実機が動いても推定位置は 0 に張り付く。
- deadband が大きいと微小な実加速度まで消える。
- 一方、deadband を抜けたバイアス誤差は二重積分で急速に増幅。
- 175852 では推定速度が約-74.9cm/s、推定 x が-150cm まで発散。
- すべて timeout で、実用的な位置閉ループには未到達。

9.3 8.3 `simple_accel_xy.py` の未コミット改良

コミット後、さらに次が追加された。

- 校正 1.5 → 2.0 秒。
- 最大出力 35 → 24。
- 最小出力 18 → 6。
- 許容誤差 8 → 5cm。
- 18cm 以内の最終低速域。
- 最終最大出力 10。
- 通常最大速度 28cm/s、最終 10cm/s。
- 速度リーク 0.75/s へ強化。
- raw IMU だけでなく姿勢角由来加速度。
- `sin(pitch 差) × 980cm/s²`
- `-sin(roll 差) × 980cm/s²`
- IMU/姿勢/blend の選択。
- 位置→目標速度→速度誤差→出力という二重ループ。
- yaw 保持。
- IMU 値、姿勢由来値、採用値をすべて CSV へ。

この変更の動機はログから強く推定できる。raw 加速度が 0 へ量子化されるため、機体の傾きから水平加速度を推定しようとした。ただし、姿勢角は「機体が傾いている」ことを示すだけで、実際の速度や風による移動を直接観測しない。さらに機体制御器が推力を変えるため、単純な $g \sin \theta$ だけでは実加速度と一致しない。

9.4 8.4 2026-07-21 inertial ログ

初期ログでは 30cm 目標に対し、推定 x が約 22.9cm で ZUPT に入る回もある一方、45.8cm へオーバーシュートし y が-24.8cm になる回もある。

未コミット改良後のログでは:

- 姿勢角が整数・粗い値で、姿勢由来加速度が 0 になる区間が多い。
- yaw が-14 度、149 度、160 度まで変化した記録がある。
- yaw 保持指令上限 12 でも大きな方位ずれを戻し切れない場合がある。
- 推定位置が 0 に留まったまま目標速度だけが出続ける回がある。

結論:

- 加速度のみは optical flow 停止時の短時間補間には使える可能性がある。
- 長時間の独立位置推定にはドリフトと量子化が大きすぎる。
- 姿勢由来加速度は raw ゼロ張り付きへの回避策だが、絶対位置観測ではない。

CHAPTER 10

9. 時間較正方式

`time_calibrated_xy.py` は、位置センサも加速度積分も使わず、「方向別に実測した速度 × 時間」で距離を作る。

既定値:

- forward: power +22、24cm/s
- back: power -22、22cm/s
- left: roll -22、22cm/s
- right: roll +22、22cm/s
- 最小セグメント 0.15 秒
- 最大セグメント 2.0 秒
- セグメント間 settle 0.25 秒
- 手動下降 throttle -12、1.2 秒

工夫:

- 前後左右を別々に較正し、非対称性を認める。
- 実測距離/時間から方向別速度を更新。
- 長距離を 2 秒以下のセグメントへ分割。
- 通常飛行 trim と着陸時 trim を分離。
- dry-run で計画だけ確認。
- optical flow 位置 API を呼ばないことを CLI 説明に明記。
- 較正、trim、着陸設定を一つの JSON に保存。

長所:

- optical flow の stale に影響されない。
- 実装が単純。
- 短い定型移動なら再現性を実測較正できる。

限界:

- 風、バッテリー、床、機体姿勢、個体差を閉ループ補正できない。
- 外乱が入っても予定時間だけ動き続ける。
- したがって、扇風機外乱に抗して直進するという最終モチベーションには単独では応えられない。

CHAPTER 11

10. コマンド積算 dead reckoning

`command_dead_reckoning.py` は optical flow を一切読まず、出した pitch/roll と経過時間から移動量を積算する。

11.1 10.1 運動モデル

- forward: 有効 pitch 1 あたり 1.30cm/s
- left: 有効 roll 1 あたり-1.15cm/s
- command deadband 4
- `dt` 上限 0.15 秒

指令値から deadband を引いた有効出力に速度係数を掛け、yaw で世界座標へ回転する。

11.2 10.2 目標制御

- 最大出力 24
- 最小出力 7
- slow radius 35cm
- 1cm 誤差あたり 0.75 command
- stop radius 6cm
- 直近 1 秒窓
- 標準偏差上限 2.5cm
- 95% 相当の上側信頼限界

停止条件は、単一時点の推定誤差ではなく、直近窓の平均、標準偏差、標準誤差から上側信頼限界を作り、それが stop radius 内かつばらつきが小さいこと。

工夫:

- 「一瞬近づいた」を到達としない。
- 推定誤差の安定性を統計的に見る。
- 世界座標目標を現在 yaw で機体座標へ変換。
- yaw 保持指令を ±15 へ制限。
- 壁位置が既知なら front ToF で x/y を blend 補正。
- 色マーカーが既知なら低高度時だけ x/y をスナップ。

11.3 10.3 実機ログ

80cm 目標の 2 回:

- 172837: 推定 x 75.15cm、誤差 4.87cm、平均 4.92cm、標準偏差 0.16cm、上側 4.98cm
- 172907: 推定 x 75.16cm、誤差 4.85cm、平均 4.90cm、標準偏差 0.16cm、上側 4.97cm

ログ上は非常に再現的に見える。しかし、これは「コマンドモデルの内部推定が再現した」ことを示すだけで、外部実測の真値とは限らない。モデルが常に同じだけ過大・過小評価すれば、統計的には安定していても実位置はずれる。

この方式の思想的価値:

- 信用できないセンサ値を無理に使わない。
- ただし、観測のない推定は真値保証ではないことを認める。
- 壁や色マーカーという疎な絶対基準が得られた瞬間だけ補正する。

CHAPTER 12

11. カラーセンサ実験

12.1 11.1 ハードウェア

CoDrone EDU 底面には 2 つの RGB カラーセンサがある。`get_colors(kind="rgb")` で両方を取得した。

12.2 11.2 実験方法

- 連続ループでカラー値を取得。
- 各取得に連番を付ける。
- 0.1 秒 sleep。
- sleep の有無・取得速度低下によるフリーズ頻度変化を体感比較。

12.3 11.3 結果

- 約 2cm 程度まで床へ近づかなければ色判定が困難。
- 通常飛行高度では床色の安定判定に使いにくい。
- 実運用では着地後、または着地直前の低高度判定。
- 約 3 秒、同じ値を返し続ける焼き付きが頻発。
- 取得速度を落としても体感頻度はほぼ同じ。
- 原因未解明。

12.4 11.4 運用上の結論

- 連続位置センサとして使わない。
- 既知の色マーカーへ低高度で接近したときの疎な位置補正には使える可能性。
- 同一値が一定時間続いた場合、それが「床色が一定」なのか「センサがフリーズ」なのか区別する鮮度判定が必要。
- 2 つのセンサが同時に同一値で停止するか、片方だけ止まるかを追加ログすると原因切り分けに有効。
- 時刻付き raw RGB、左右センサ別、変化量、連続同値時間を保存すべき。

CHAPTER 13

12. 球を風で 30cm 動かす実験

13.1 12.1 目的

前方 10cm に置いた直径約 7cm の穴あきプラスチック製 pickleball を、接触せず、ドローンの風で 30cm 移動させる。

13.2 12.2 条件

- 床: グレーのフローリング
- 配置基準: 離陸前、機体先端から球の手前端まで 10cm
- バッテリー最低 40%
- 安全高度 25cm
- 初期 pitch 10
- throttle 0
- 風パルス 0.50 秒
- 1 回
- 前後 hover 2.0 秒 / 1.0 秒
- sample 周期 0.1 秒

13.3 12.3 安全上の工夫

- 離陸前だけ front ToF で球が 6〜18cm 相当に見えるか確認。
- 900cm 相当なら未検出として中止。
- 球が近すぎ・遠すぎでも中止。
- 離陸後は床上の球が front センサ視野から外れるため、front range を安全判定に使わない。
- 高度 25cm 未満、または高度取得不能で中止。
- 各パルス後に全指令を 0。
- 例外や中断でも停止・着陸・close。

13.4 12.4 調整方法

一度に主変数を一つだけ変更する。

1. `blow_duration_sec`
2. `blow_pitch_power`
3. `blow_repeats`
4. `target_front_clearance_cm`

30cm 未満ならまず時間を +0.15 秒、超過なら-0.10 秒。時間が長すぎても不足なら pitch +2。短時間でも超過なら pitch -2。

考え方:

- 時間は姿勢への影響が比較的小さく、最も線形に調整しやすい。
- pitch は効くが、機体自身が球へ近づく危険がある。
- 強い一発より弱い複数パルスで摩擦ばらつきを平均化できる可能性。
- 1 回で結論を出さず、最低 3 回、最終的に 5 回中 4 回が 27〜33cm を採用条件とする。

13.5 12.5 現存ログの状態

7 つの JSONL がある。後半 4 ログは風パルスと summary まで存在するが、`actual_ball_move_cm` はすべて未入力である。

したがって:

- コードと安全運用の試験は行われた。
- しかし球の実測移動量が記録されていない。
- 現時点ではどの pitch/時間が 30cm に適切か評価できない。
- 自動提案も「実測値を記録してから判断」で止まっている。

CHAPTER 14

13. 離陸診断

`codrone_takeoff_diagnose.py` は、takeoff が成功したかを高さだけで判断せず、内部状態を時系列保存する。

取得:

- battery
- state data
- error data
- count data
- height
- bottom range
- accel x/y/z

デコード:

- LowBattery
- Propeller/Motor failure
- Vibration
- Attitude not stable
- Motion sensor no answer / wrong / not calibrated
- Pressure sensor error
- Ground range error
- Flow no answer / wrong / ground image recognition failure
- flight state

手順:

1. pair 後
2. move 値 reset 後
3. takeoff 直後
4. 0.5 秒周期の hold 中
5. JSON 保存
6. land/close

モチベーション:

- 高度センサが 0 でも飛行状態は Flight/Hovering の場合がある。
- 「離陸できなかった」を一つの API 戻り値だけで判断せず、状態・エラー・カウンタ・実高度を組み合わせ

たい。

- optical flow や下方 ToF 自体が壊れているとき、高度 0 をそのまま離陸失敗扱いすると誤診する。

CHAPTER 15

14. テストしたこと

15.1 14.1 現在の自動テスト結果

2026-07-27 に以下を実行した。

```
python -m unittest -v
```

結果:

- 35 テスト
- すべて成功
- 実行時間約 0.57 秒

また、プロジェクト内の全 Python ファイルを `py_compile` し、構文エラーなしを確認した。

15.2 14.2 `test_drone_precision.py`

15.2.1 停止距離

- 物理制動距離にセンサ遅延と余裕距離が含まれる。
- 静止時は余裕距離だけになる。

15.2.2 状態推定

- 速度計算が制御周期ではなく、新鮮な位置パケット間隔を使う。

15.2.3 座標変換

- yaw 0 で軸が保存。
- yaw 90 度で世界 x が機体横方向へ回転。
- world → body → world が元へ戻る。

15.2.4 固定符号

- yaw 符号が実機観測値`-1` と一致。

15.2.5 PID

- 近傍で最小出力を強制しない。

- 出力飽和中に積分が増えない。

15.2.6 settle 指令

- 遠く低速なら目標方向へ。
- 近く高速ならブレーキ。
- D/速度項が早すぎる逆指令を出しても、目標方向へ修正。

15.2.7 高速状態

- 長距離 x で最大 pitch 加速と最大制動。
- 長距離 y で最大 roll 加速と最大制動。
- z でも throttle 高速フェーズ。

15.2.8 ドリフトガード

- 振幅が増える符号反転系列で水平出力縮小。

15.3 14.3 test_accel_position_control.py

- 校正が raw 加速度平均を使う。
- raw 単位換算。
- 一定加速度の位置積分。
- deadband による小ノイズ除去。
- x 軸符号反転。
- 正 x 目標が正 pitch。
- 正 y（左）が負 roll。
- 出力上限。
- 現場確認用の指令符号反転。
- CLI 既定で、実機ログに合わせ x 加速度を反転。
- 高度 0 でも flight/movement state が飛行中なら離陸成功。
- state error があれば失敗。

15.4 14.4 test_time_calibrated_xy.py

- 方向別の正しい符号。
- 長い移動をセグメント分割。
- 実測距離/時間から速度更新。
- trim 更新で移動較正を壊さない。
- 着陸設定更新で takeoff trim を壊さない。
- 正の着陸 throttle を拒否。

15.5 14.5 テストが保証しないこと

- 実際に 30cm 進むこと。
- 扇風機の風に勝てること。
- optical flow が正しいこと。
- カラーセンサがフリーズしないこと。
- 加速度積分が長時間正しいこと。
- 上昇と下降の実機応答。
- バッテリー、床、照明、個体差。

自動テストは、実機失敗から得た「数式上・コード上、二度と壊したくない条件」を固定するものと位置づけるべきである。

CHAPTER 16

15. 実験方法上の工夫

16.1　15.1 短距離で符号を確認

2,000mm で符号を間違えると大きく離脱する。そこで 300mm、さらに 100mm へ縮小し、初動 1 秒の向きを確認した。

16.2　15.2 一次原因を先に直す

ログでは複数の問題が同時に見える。優先順位は概ね次だった。

1. 符号が正しいか。
2. 基準位置が新鮮か。
3. センサが更新しているか。
4. 高度・yaw が安全か。
5. その後にゲイン・最速化。

符号が逆なのに PID ゲインを調整しても意味がない。

16.3　15.3 仮説を撤回する

100mm の一回から「符号ではなく stale」と判断したが、次の低 stale 逆走ログで撤回した。一回の都合のよい結果に固執せず、反証ログを優先した。

16.4　15.4 回数ではなく実時間

stale 18 cycles という閾値は制御周期依存だった。0.06 秒周期でも位置更新はそれより遅い。そこで 0.30 秒で出力抑制、0.65 秒で停止など、実時間へ変更した。

16.5　15.5 異常ログを自動学習から除外

runaway やセンサ異常を学習すると、壊れた飛行に合わせてゲインが変わる。正常終了または一定条件の timeout だけを調整対象とした。

16.6　15.6 全周期ログ

位置だけでなく、速度、誤差、指令、積分、fresh/stale、制御フェーズ、予測残距離、停止距離、終了理由を保存した。これにより「最終誤差が悪い」から一歩進み、「いつ、何を根拠に、どの指令を出したか」を追える。

16.7 15.7 一度に一変数

球風実験では、時間、pitch、回数、距離を同時に変えない。同一条件を最低 3 回試し、再現性を見てから次へ進む。

16.8 15.8 人間の実測を正式なデータとして扱う

球位置はセンサだけでは測れないため、手動測定値を後から JSONL と CSV へ追記し、その値から次設定を提案する設計にした。

16.9 15.9 安全処理を通常ロジックと同格に扱う

- battery check
- timeout
- 出力制限
- zero commands
- hover
- land
- close
- exception/finally
- runaway
- sensor timeout
- altitude fault
- attitude fault

これらは付加機能ではなく、最大出力・急制動実験を可能にする前提である。

CHAPTER 17

16. ファイル別の役割

17.1 16.1 現在の中核

- drone_precision.py: optical flow 位置を使う多軸ハイブリッド制御。
- main.py: 共通 CLI 入口。
- pid_test1.py: 同じ共通 CLI への互換入口。
- test_drone_precision.py: 回帰テスト。

17.2 16.2 optical flow 代替

- simple_accel_xy.py: IMU/姿勢由来加速度と ZUPT。
- accel_position_control.py: 構造化した加速度専用推定・制御。
- time_calibrated_xy.py: 方向別実測速度 × 時間。
- command_dead_reckoning.py: 指令積算、壁・色補正、統計停止。

17.3 16.3 診断・個別実験

- codrone_takeoff_diagnose.py: 離陸失敗診断。
- ball_wind_experiment.py: 球への風パルス。
- sensor_1.py: カラーセンサ連続取得。
- front_range.py: 前方 ToF。
- height.py: 下方距離・高度。
- barometer.py: 気圧。
- Gyro_sensor.py: 姿勢角。
- battery.py: バッテリー。
- temperature.py: 温度。

17.4 16.4 初期飛行・手動操作

- xy-axis.py: 固定時間の前後左右。
- z-axis.py: 強い上下指令の試作。
- keyboard_input.py: 文字入力による手動飛行。
- move.py: キーボードで目標座標を動かす初期 x/y PID。
- z-pid.py: 初期高度 PID。
- time-based_flight.py: 指定秒飛行。
- 30sec.py: 30 秒飛行と時間誤差観察。
- flight_time.py: 離着陸時間。

- function.py: 正方形飛行。
- pentagon.py: 五角形飛行。
- 3_1_sine.py: 上下交互運動。
- brake.py: 着陸指令確認の小スクリプト。
- user_input.py: 入力回数に応じた回転。

17.5 16.5 非飛行・周辺 API

- Westminster_Chimes.py
- buzzer.py
- doremifasollasido.py
- random_noise.py
- shine.py
- practice_random.py
- function.py

これらは最終制御に直接使われないが、通信、ブザー、LED、ループ、関数化、random、入力といった初期学習の痕跡である。

17.6 16.6 文書

- memo/tech_spec.md: 機体仕様と最終目標。
- memo/requirements.md: できること・できないこと・完成条件。
- memo/control_spec.md: ハイブリッド制御方式。
- memo/implementation_guide.md: 実行・実験・テスト手順。
- experiment_logs/解析方法.md: 毎回同じ観点でログを見る手順。
- experiment_logs/実験履歴サマリー.md: 生ログからの数値評価。
- 日付別実験記録: 変更理由、結果、次回項目。

CHAPTER 18

17. 当時の思想・考え方の変遷

18.1　17.1「動けばよい」から「説明できなければならない」へ

初期コードは API を呼んで機体が動くことが目的だった。後期コードでは、各出力がなぜ出たかをログから説明できることが要件になった。

18.2　17.2「同じ命令」から「同じ結果」へ

固定時間制御は同じ命令を再生できる。しかし求めているのは同じ最終位置である。外乱がある以上、命令の再現ではなく結果のフィードバックが必要になった。

18.3　17.3「PID なら解ける」から「観測できるものしか制御できない」へ

PID は誤差が正しく観測できることを前提とする。optical flow が滑り、カラーがフリーズし、加速度が量子化されるなら、ゲインを上げても真値には近づかない。要件文書の「観測できない絶対位置は制御できない」は、このプロジェクトで最も重要な思想的到達点である。

18.4　17.4「平均性能」から「失敗時の安全」へ

最速化を目指しながら、runaway を早期終了し、高度急落時に水平移動を捨てる。これは成功時の速さより、失敗が大事故にならないことを前提にした設計である。

18.5　17.5「単一正解」から「複数モデルの使い分け」へ

- optical flow: 位置を直接出すが破綻し得る。
- IMU 積分: 高周波補間できるがドリフトする。
- 時間較正: 簡単だが外乱を見ない。
- コマンド積算: 滑らかだがモデル誤差を見ない。
- front ToF: 壁があるときだけ絶対補正。
- color: 低高度・既知マーカー時だけ絶対補正。

後期の方向性は、どれか一つを万能とせず、使える場面だけ使うことである。

18.6　17.6「最速」の意味の具体化

単に出力 100 を出すことではない。

- 遠方: 安全に最大加速。

- 停止距離: 最大制動。
- 近傍: 速度サーボ。
- yaw ずれ: 先に向きを直す。
- 高度低下: 先に高度を戻す。
- センサ停止: 速さを捨てて止まる。

総所要時間を最小化するには、暴走・やり直し・振動を減らす必要がある。

CHAPTER 19

18. 未解決事項

19.1 18.1 optical flow

- stale の根本原因。
- 床材・照明・高度・傾斜別の定量評価。
- 高速時にどの条件で破綻するか。
- 同じ値が本当に静止なのか、更新停止なのかの識別。

19.2 18.2 カラーセンサ

- 3 秒焼き付きの原因。
- 左右 2 センサが同時に止まるか。
- raw RGB と色名変換のどちらで止まるか。
- 高さごとの判定精度曲線。
- 照明、床反射、色面積の影響。

19.3 18.3 上下制御

- 上昇・下降別の独立ゲイン。
- 下降時の最小推力、地面効果、センサ遅延。
- throttle 0 が「高度維持」なのか「内部制御任せ」なのか。
- 下向き速度の実測。

19.4 18.4 加速度推定

- raw 単位換算の実機精度。
- 軸符号と機体姿勢。
- センサ量子化。
- 温度・振動によるバイアス変化。
- ZUPT 誤発動。
- 姿勢角由来加速度の妥当性。
- 短時間補間へ限定した sensor fusion。

19.5 18.5 時間較正・dead reckoning

- 実測位置との比較が不足。

- バッテリー別速度。
- 前後左右非対称性。
- 風を受けたときのモデル誤差。
- wall/color 補正の実機検証。

19.6 18.6 球風実験

- 実測移動量未入力。
- 30cm 達成条件不明。
- 横ずれ、接触、バッテリー差の記録不足。

19.7 18.7 ソフトウェア

- 未コミットの大規模コードとログの整理。
- `brake.py` の別インスタンス問題。
- `z-axis.py` の `for i in 10`。
- 初期ファイルの文字化けコメント。
- hardware-in-the-loop テスト。
- 実験条件メタデータの統一。

CHAPTER 20

19. 現在地点の正確な評価

20.1 達成したこと

- 基本 API と各センサの個別確認。
- 時間制御、手動制御、単軸 PID、多軸 PID。
- ログ、summary、履歴解析、距離別プロファイル。
- 世界/機体座標変換。
- yaw 優先。
- 最大加速・最大制動・近傍速度サーボ。
- stale 実時間判定。
- runaway、高度、姿勢、センサ安全停止。
- optical flow 非依存の 4 系統の試作。
- 35 件の回帰テスト。
- 実験記録と仕様書。
- カラーセンサの実用高度と焼き付き問題の発見。

20.2 未達成

- x/y/z の指定量を、実機で毎回、高速かつ高精度に再現。
- 風・物理妨害後の厳密な復帰。
- optical flow 破綻後の真位置復元。
- 上昇・下降の非対称性を完全にモデル化。
- カラーセンサのフリーズ原因解明。
- 球 30cm 移動の定量的成功。

20.3 現実的な結論

内蔵センサだけで、センサが健全な短時間・適切な床・照明・高度の範囲なら、相対移動を改善できる可能性はある。しかし、センサが真位置を失った後も「厳密な絶対位置」を保証することはできない。これは制御ロジックの努力不足ではなく、観測可能性の限界である。

CHAPTER 21

20. 次に行うべき実験

21.1　20.1 センサを同時記録する

同一飛行で以下を同時に記録する。

- optical flow x/y
- raw accel x/y/z
- roll/pitch/yaw
- gyro
- bottom ToF
- barometer
- color 左右 raw RGB
- 指令 pitch/roll/throttle/yaw
- バッテリー
- monotonic timestamp

目的は、どのセンサが先に破綻するかを比較すること。

21.2　20.2 外部測定は評価専用に使う

最終制御に外部測位を使わない制約を維持しても、性能評価だけは床のメジャー、動画、上方カメラなどで真値を取るべきである。内部推定値だけで内部推定器を評価すると、dead reckoning のように自己整合的だが真値からずれた結果を成功と誤認する。

21.3　20.3 上昇・下降を別実験にする

- 同じ高度差。
- 同じバッテリー。
- 上昇と下降を別々に 10 回。
- 指令、速度、停止距離、オーバーシュート。
- ゲインを共有しない。

21.4　20.4 カラー焼き付きの切り分け

- 左右 raw RGB を個別取得。
- API 戻り全体と個別チャンネルを比較。
- 10Hz、5Hz、2Hz、1Hz。

- 静止色と色切替を一定周期で行う。
- 同値継続時間と、実際の色変更時刻を記録。
- 再 pair、センサ reset、API 再呼出しで復帰するか。

21.5 20.5 代替推定の役割分担

- optical flow: 通常位置。
- IMU: 0.2〜0.5 秒程度の短時間補間。
- command model: stale 中の予測。
- bottom ToF/barometer: z。
- wall ToF: 機会的な絶対補正。
- color marker: 着地直前の位置スナップ。

加速度積分を単独で長時間使わず、stale ギャップを埋める補助へ限定するのが合理的である。

CHAPTER 22

21. 最終総括

このプロジェクトで行われたことは、PID ゲインを何度か変えた、という一言では表せない。

最初は離陸、着陸、音、LED、前後左右を一つずつ確認した。次に、時間で動かすだけでは再現しないことに直面し、座標誤差を戻す PID へ進んだ。PID では、積分飽和、整定、出力上限、バッテリー、例外着陸を考えた。実機ログを取り始めると、y の大発散、古い基準位置、高度 0 誤認、pitch 符号逆転、stale、上下振動、yaw ずれが別々の問題として現れた。

そこで、短距離化による符号確認、仮説の撤回、stale 回数から実時間への変更、異常ログの学習除外、世界座標から機体座標への変換、yaw 優先、runaway、高度回復を導入した。さらに、最速を本気で求めるなら純 PID だけでは足りないと判断し、最大加速、物理停止距離、最大制動、近傍速度サーボへ移った。

しかし、その制御が依存する optical flow 自体が不安定だった。そのため加速度積分、姿勢由来加速度、ZUPT、時間較正、コマンド積算、壁補正、カラーマーカー補正という別の手法を考えた。これらのログから、加速度はゼロ張り付きとドリフト、時間制御は外乱非観測、dead reckoning は真値非保証、カラーは 2cm 制約と 3 秒焼き付きという、それぞれ別の限界が見えた。

この過程を貫くモチベーションは一貫している。

> 見た目にたまたま成功する飛行ではなく、なぜその指令を出し、なぜ成功または失敗し、外乱があってもどこまで戻れ、戻れない場合はなぜ戻れないかを説明できる制御を作りたい。

現在、ソフトウェア構造、ログ、テスト、安全ガード、実験方法はかなり成熟した。一方、実機の最終精度と再現性はまだ完成していない。最大の障壁はゲイン調整そのものではなく、内蔵センサから真の位置をどこまで観測できるかである。

したがって今後の中心課題は、「さらに複雑な PID」を作ることではない。各センサの有効範囲と故障モードを定量化し、短時間 IMU 補間、通常時 optical flow、機会的な壁・色補正を組み合わせ、それでも観測不能になったときは安全に不確実性を認めて停止することである。

CHAPTER 23

22. 結論・事実・原因・対応策を対応づけた総合判定

この節は、報告書全体の内容を「結論だけ読んだ後、何を根拠にその結論へ至ったか」を逆引きできるようにしたものである。各項目について、確認できた事実、原因、原因の確度、すでに行った対応、今後必要な対応を分離する。

23.1　22.1 同じプログラムでも飛行結果の再現性が低い

結論

CoDrone EDU へ同一の指令列を与えるだけでは、同一の移動結果を再現できない。最終目標を「命令の再生」ではなく「観測された誤差への継続補正」とした判断は妥当である。

確認事実

- 2026-06-23 の同系統 PID ログで、x 最終誤差は`-86.7mm` から `+87.0mm` まで変化した。
- y 最終誤差は `+106.3mm` 程度の回だけでなく、`-1008.8mm`、`-3274.7mm` まで崩れた回がある。
- 初期群の最良回は `x-3.9mm`、`y-18.0mm`、`z-2.9mm` だったが、同程度の成功が継続しなかった。
- 利用者は、飛行中にあらゆる方向へのぶれが激しく、同じプログラムでも結果の再現性が低いと明示している。

原因

複数原因が重なっている。

- 風や空調などの外乱。
- バッテリー電圧・残量による推力差。
- 離陸直後の姿勢差。
- pitch/roll/yaw/throttle の機体応答の非線形性。
- optical flow の更新遅延・停止・誤差。
- yaw ずれによる制御座標の回転。
- 高度と床画像条件の変化。
- 出力符号、ゲイン、最小出力、制動開始時刻の不整合。

これらのうち、ログで直接確認できたのは位置更新停止、yaw ずれ、符号逆転、高度変動、速度・誤差の発散である。風、床、バッテリー差の寄与率は現存ログだけでは分離できない。

行った対応

- 時間制御から位置閉ループ制御へ移行。

- x/y/z/yaw を同時監視。
- yaw 座標変換。
- stale 監視。
- 最大加速・制動・近傍速度サーボ。
- 直近誤差の増幅を監視するドリフトガード。
- 実験条件と全周期ログの保存。

今後必要な対応

- 同条件反復回数を最低 5〜10 回へ。
- バッテリー、床、照度、風速、離陸位置、機体向きをメタデータ化。
- 内部推定とは独立した外部真値で評価。
- 平均誤差だけでなく標準偏差、最大誤差、失敗率を評価。

23.2 22.2 純粋な PID だけでは最速・精密・堅牢を同時達成しにくい

結論

純 PID は最終位置の微調整には使えるが、遠距離の最高速加速、センサ遅延を考慮した制動、異常状態の処理を単独で表現するには不十分である。状態機械と速度・停止距離モデルを主とし、PID を補助にする設計へ移ったことは、ログに基づく合理的な変更である。

確認事実

- 旧 PID ログは多くが約 20 秒、後期ログは約 35 秒の timeout まで使った。
- 目標近傍でも高い速度が残り、オーバーシュートや符号反転が起きた。
- 最小出力が目標近傍でも指令を強制し、上下振動を増やした。
- PID の D 項・速度項が、まだ十分遠い段階で逆向き指令を出す場合があった。
- 高速移動中の停止には、現在誤差だけでなく現在速度とセンサ遅延が必要だった。

原因

PID は基本的に現在・過去の誤差から出力を作る。物理的に必要な停止距離、センサパケットが次に来るまでの進行量、加速・巡航・制動という明示的な飛行フェーズを直接表現しない。ゲインだけでそれらを暗黙に実現しようとすると、短距離では強すぎ、長距離では遅すぎるという矛盾が生じる。

行った対応

- x/y は 500mm 以上、z は 300mm 以上で高速フェーズを許可。
- `acquire` → `accelerate` → `brake` → `settle` を導入。
- $v^2/(2a)$ + v×latency + margin で停止距離を計算。
- 近傍では残距離から到達可能速度を求め、速度サーボ。

- 制動条件では PID 出力より逆向きブレーキを優先。
- 最小出力を近傍では無効化。

今後必要な対応

- 実機から軸別・方向別の実減速度を測定。
- 上昇・下降、前進・後退、左右で別の停止モデルを持つ。
- 距離別に状態遷移の成功率を測る。
- 最大出力を使う前にセンサ有効条件を判定する。

23.3 22.3 optical flow は最終目標に対する単独の信頼源にならない

結論

optical flow sensor は、条件が良いときの短時間相対位置には利用価値があるが、外乱下で厳密な位置保証を行う単独センサとしては不十分である。利用者の「カス」という評価は、絶対的に無価値という意味ではなく、要求水準に対して支配的な弱点になったという意味で理解すべきである。

確認事実

- stale 比率 0.70〜0.81 のログが複数ある。
- 古い位置・高度 0 を開始基準として採用した失敗がある。
- 更新再開後の位置差を短い制御周期で割り、異常に大きい速度を作った。
- 高速・傾斜・yaw 変化を伴うログで位置誤差が大きく崩れた。
- 仕様書にも床、照明、高度、傾斜への依存と、滑った後の絶対位置復元不能が明記されている。

原因

- 下向きカメラが床画像の移動を追うため、特徴の少ない床や照明変化に弱い。
- 機体が傾くと画像の見え方と高度が変わる。
- 高速移動ではフレーム間変位が大きくなる。
- センサ更新周期と Python 制御周期が一致しない。
- 内部 API が同じキャッシュ値を返しても、それだけでは静止と stale を区別できない。

床画像や内部ファームウェアの詳細ログがないため、各ログの stale が画像認識失敗、通信遅延、API キャッシュのどれに由来するかは確定できない。

行った対応

- fresh 判定。
- stale cycle から stale 実時間へ。
- stale 中の出力制限。

- 新鮮な位置パケット間隔による速度計算。
- 基準位置を複数サンプルで取得。
- 加速度、時間較正、command dead reckoning を代替案として実装。

今後必要な対応

- flow error flag と位置更新を同じ時刻で保存。
- 床材・照度・高度・傾斜を変えたベンチ試験。
- stale 中は IMU で短時間補間し、不確実性を時間とともに増やす。
- 一定時間を越えたら推定を信用せず停止。

23.4 22.4 加速度二重積分だけでは長時間位置を維持できない

結論

加速度だけの位置推定は、optical flow の短い欠落を埋める補助にはなり得るが、現状の CoDrone EDU raw 加速度を二重積分して長時間の x/y 絶対位置を作る方式は実用精度に達していない。

確認事実

- 30cm または 60cm 目標に対し、推定位置 0cm のまま timeout したログがある。
- 別ログでは 30cm 目標に対し推定位置`-150.3cm`、最終誤差 `180.3cm`。
- raw x/y が 0 に張り付く区間がある。
- 小さなバイアス差が速度・位置へ累積した。
- 既存 4summary はすべて timeout。

原因

- 加速度の量子化。
- 静止校正後にも残るバイアス。
- モータ振動。
- 機体座標と世界座標の回転。
- 重力成分の混入。
- deadband が大きいと実信号を消し、小さいとノイズを積分。
- 速度リークはドリフトを抑える一方、実速度も失わせる。
- ZUPT は停止判定を誤ると移動中速度まで 0 にする。

行った対応

- バイアス平均。
- LPF。

- deadband。
- dt 上限。
- 速度リーク。
- 台形積分。
- yaw 座標変換。
- gyro・加速度・小指令を使う ZUPT。
- raw IMU が 0 になる問題に対し、姿勢角由来加速度と blend を追加。
- 位置直接 P 制御から、位置→目標速度→速度制御へ変更。

今後必要な対応

- 単独航法ではなく、0.2〜0.5 秒程度の短い stale 補間へ限定。
- 推定共分散・信頼度を持つ。
- optical flow 更新時に緩やかに再同期。
- 外部真値と比較して補間時間ごとの誤差を定量化。

23.5 22.5 カラーセンサは通常飛行高度の連続位置観測に適さない

結論

底面カラーセンサは、通常飛行中の床色追跡ではなく、着地直前または着地後の色判定、あるいは既知マーカー上での疎な位置補正へ用途を限定すべきである。

確認事実

- 約 2cm 程度の高さでなければ色を判定しにくかった。
- 約 3 秒、同じ値を返し続ける焼き付きが頻発した。
- sleep で取得頻度を落としても体感頻度は同程度。
- 2 つの RGB センサを取得するコードへ変更された。

原因

- 高度が上がると検出対象の反射光が弱くなる。
- センサ視野へ複数色・床反射が混ざる可能性。
- API またはファームウェア側のキャッシュ。
- センサ更新・通信・色変換処理のいずれかの停止。

最後の 3 点は仮説であり、現存コードは時刻付き raw 左右値をファイルへ保存していないため確定できない。

行った対応

- 取得周期を sleep で変更してポーリング過多説を確認。

- command dead reckoning では、色補正を高度 18cm 以下の既知マーカー時だけ適用する設計にした。

今後必要な対応

- 左右別 raw RGB、色名、時刻、連続同値時間をログ。
- 実際に色を切り替える既知時刻と応答を比較。
- 1Hz〜20Hz で更新率比較。
- reset、再 pair、API 再生成で復帰するか確認。

23.6 22.6 上昇と下降は対称ではない

結論

z 軸を一つの対称な PID と一つの加速度定数で扱うべきではない。上昇と下降は別のゲイン、出力下限、速度限界、停止距離を持つべきである。

確認事実

- 利用者は上昇は得意、下降は苦手と観察している。
- 171035 では目標近傍で throttle が正負反転し、上下運動を繰り返した。
- 172317 では z 最終誤差が約 798mm、z 速度ピーク約 741mm/s。
- 高度回復ガードが後に追加された。

原因仮説

- 上昇は推力増加で直接作れるが、下降は重力と残留推力の差で決まり応答が遅い。
- 飛行制御器が内部で高度・姿勢を補償している可能性。
- 下方 ToF・barometer の遅延。
- 地面付近のダウンウォッシュ・地面効果。
- 水平傾斜による鉛直推力減少。

行った対応

- throttle 最小出力を 0。
- 近傍ゲインと出力上限を段階低下。
- z 速度ブレーキ。
- 小誤差・低速時に throttle 0。
- 水平出力に応じた tilt compensation。
- 高度急落時に水平制御を捨て throttle 80。

今後必要な対応

- 上昇・下降を分けて実減速度を測定。
- `brake_accel_up` と `brake_accel_down` を分離。
- 高度センサごとの遅延を比較。
- 地面から十分離れた条件と低高度条件を分ける。

CHAPTER 24

23. PID 飛行ログ全 42 件の完全一覧

24.1 23.1 表の読み方

この一覧は現存する `flight_logs/pid_flight_*.jsonl` 全 42 件を対象とする。

- `samples`: `sample` レコード数。
- `reason`: summary に保存された終了理由。
- —: 値が 0 という意味ではない。古いログ形式に項目が存在しない、または summary 自体がない。
- `ex/ey/ez`: 最終位置誤差。正負は目標のどちら側に残ったかを示す。絶対値が精度を示す。
- `stale`: 位置サンプルが fresh でなかった比率。
- `peak x/y/z`: 推定された軸速度の最大絶対値。
- `osc x/y/z`: 誤差符号反転回数を中心にした振動指標。センサ飛びも含む可能性があるため、純粋な物理振動回数とは限らない。

古い 2026-06-23 ログには `termination_reason`、`stale_ratio`、軸別 peak、軸別 oscillation が保存されていない。これは当時それらを評価していなかったという意味ではなく、ログ形式が後から拡張されたという意味である。

ログ	samples	reason	duration[s]	ex[mm]	ey[mm]	ez[mm]	stale	peak x	peak y	peak z	osc x	osc y	osc z
20260623_164713	316	—	19.98	72.30	27.84	22.27	—	—	—	—	—	—	—
20260623_165545	208	summary なし	—	—	—	—	—	—	—	—	—	—	—
20260623_170224	321	—	19.95	81.32	-99.63	26.27	—	—	—	—	—	—	—
20260623_170643	319	—	19.99	-13.84	-3274.68	10.59	—	—	—	—	—	—	—
20260623_171101	321	—	20.00	87.02	-1008.81	13.22	—	—	—	—	—	—	—
20260623_171450	240	—	14.96	50.08	106.28	-17.75	—	—	—	—	—	—	—
20260623_172036	307	—	19.95	31.67	89.42	-12.42	—	—	—	—	—	—	—
20260623_173526	321	—	19.99	-86.74	3.09	-20.67	—	—	—	—	—	—	—
20260623_173618	318	—	19.96	26.65	-41.75	-26.34	—	—	—	—	—	—	—
20260623_174318	212	—	13.29	-3.94	-17.97	-2.92	—	—	—	—	—	—	—
20260630_163702	31	—	1.88	1967.11	-35.74	-759.72	0.677	98.87	657.75	5307.76	0	0	0
20260630_164659	168	—	10.46	2424.35	-47.38	63.01	0.702	513.77	127.34	195.62	0	3	0
20260630_165510	239	—	15.00	1370.06	-67.31	82.86	0.762	574.70	335.33	198.53	0	3	1
20260630_165714	0	summary なし	—	—	—	—	—	—	—	—	—	—	—
20260630_165807	152	—	9.47	659.41	69.23	10.79	0.724	320.03	143.30	224.78	0	0	0
20260630_170026	72	—	4.48	45.47	1.15	26.05	0.806	174.64	64.40	85.91	0	2	0
20260630_170651	243	—	19.96	2415.33	-51.50	63.94	0.037	951.47	115.03	104.04	0	0	2
20260630_171035	243	—	19.99	64.16	-44.60	26.56	0.128	62.75	62.70	78.95	0	3	0
20260630_172047	244	—	20.00	27.11	-7.92	51.88	0.066	509.23	112.72	103.35	0	1	0
20260630_172711	242	—	19.96	1065.89	16.27	34.81	0.727	311.96	84.83	79.87	0	4	0
20260630_174211	220	runaway	18.13	72.84	35.76	0.57	0.727	74.82	45.05	23.36	0	2	8
20260707_164639	53	runaway	4.27	-27.90	-13.20	70.26	0.358	345.84	114.14	60.58	1	4	0
20260707_164950	26	runaway	2.07	301.85	-83.14	38.19	0.346	652.28	68.69	127.23	2	0	0
20260707_165019	21	runaway	1.67	76.19	232.50	83.90	0.381	88.15	366.67	113.12	0	0	0
20260707_165327	13	runaway	0.99	168.20	46.80	47.02	0.385	395.30	54.98	73.23	0	0	0
20260707_165345	9	runaway	0.66	-63.84	527.21	43.24	0.333	117.91	623.77	77.30	0	0	0
20260707_165911	22	runaway	1.73	185.47	-66.93	60.10	0.364	382.30	67.52	76.11	0	0	0
20260707_165932	23	runaway	1.84	159.41	117.08	28.73	0.348	165.58	496.02	71.94	0	0	0
20260707_170225	22	runaway	1.91	223.72	-85.78	82.57	0.273	336.57	156.95	101.26	0	0	0
20260707_170244	39	runaway	3.15	132.40	350.24	85.39	0.359	247.58	198.12	230.88	0	0	0
20260707_170506	26	runaway	2.13	405.99	-7.89	76.29	0.385	273.35	46.41	151.38	0	2	0
20260707_170526	31	runaway	2.47	-45.09	382.74	125.75	0.355	146.79	470.18	80.06	0	0	0
20260707_170935	242	timeout	19.95	-33.63	60.96	62.78	0.326	327.29	124.46	77.36	5	0	0
20260707_171019	229	timeout	19.97	-16.73	47.19	46.39	0.323	87.72	250.01	90.40	6	2	2
20260707_171632	410	timeout	34.95	-57.51	-60.68	62.13	0.332	253.83	149.45	98.52	9	11	0
20260707_171724	424	timeout	34.99	66.72	-71.17	53.99	0.344	71.01	472.48	101.18	9	3	0
20260707_172317	425	timeout	34.94	43.42	-75.96	798.35	0.376	328.80	69.41	741.09	12	6	0

次ページへ続く

ログ	samples	reason	duration[s]	ex[mm]	ey[mm]	ez[mm]	stale	peak x	peak y	peak z	osc x	osc y	osc z
20260707_172418	401	timeout	34.93	-4.65	69.45	29.96	0.359	58.54	257.71	61.09	1	4	0
20260707_173145	422	timeout	34.94	18.33	-81.41	-20.37	0.329	411.57	336.80	55.53	26	26	5
20260707_173239	410	timeout	34.97	127.85	-1.17	7.53	0.332	529.70	421.60	126.72	12	25	30
20260707_173920	14	atti-tude_fault	1.08	191.20	98.23	35.08	0.286	157.59	144.08	35.90	0	2	0
20260707_173949	418	timeout	34.93	109.01	-82.57	-3.12	0.335	138.61	242.31	77.59	4	27	27

24.2 23.2 全ログを一件ずつ言語化した評価

24.2.1 20260623_164713

約 20 秒制御しても x に 72mm、y に 28mm、z に 22mm 残った。三軸とも数 cm 級の偏差があり、到達精度不足である。ただし後の数 m 級発散と比較すれば、初期基準としては大崩れしていない。

24.2.2 20260623_165545

208 サンプルは存在するが summary がない。プログラムの途中中断、例外、summary 実装前の終了などが考えられるが、ログだけでは理由を確定できない。最終結果の比較対象から除外すべきである。

24.2.3 20260623_170224

x81mm、y 約-100mm、z26mm。x だけを主目標とした制御でも横流れが約 10cm に達している。副軸を監視する必要性を示す。

24.2.4 20260623_170643

x 誤差は-14mm と一見良いが、y が-3275mm である。x だけを成功判定に使うと危険な誤評価になる代表例である。主軸の精度と機体全体の安全は別である。

24.2.5 20260623_171101

y が約-1009mm。前ログより小さいが、依然として 1m 級の横発散。roll 符号、横位置センサ、yaw、軸間干渉のいずれか、または複数が支配的だった。

24.2.6 20260623_171450

約 15 秒で終了し、x50mm、y106mm、z-18mm。時間は短いが、横誤差が大きい。速く終わることと正しく到達することは同義ではない。

24.2.7 20260623_172036

x32mm、y89mm、z-12mm。高度は比較的良いが、y が約 9cm 残った。初期群で y 制御が継続的弱点だったことを補強する。

24.2.8 20260623_173526

y3mm と非常に良い一方、x は-87mm、z-21mm。ある軸を改善すると別軸が悪化する軸間トレードオフが見える。

24.2.9 20260623_173618

x27mm、y-42mm、z-26mm。突出した発散はないが、全軸許容内には入っていない。

24.2.10 20260623_174318

x-3.9mm、y-18.0mm、z-2.9mm、13.29 秒。初期群の最良。単発性能としては高いが、前後ログとのばらつきが大きいため、制御方式の再現成功とは判断できない。

24.2.11 20260630_163702

1.88 秒で終了し、x 約 1967mm、z 約-760mm。z 速度ピーク 5308mm/s は物理的な真速度というより、古い 0 高度から実高度へ値が飛んだことを短時間差分した推定異常を強く示す。基準位置取得失敗と stale 停止の問題を発見したログである。

24.2.12 20260630_164659

基準高度は正常化したが x 誤差 2424mm。stale 70% に加え、主進行方向が逆。センサ問題だけでなく符号問題を疑う契機になった。

24.2.13 20260630_165510

固定符号後も x 誤差 1370mm。300mm 級の短距離へ縮小しても逆方向へ進んだため、「長距離だから難しい」のではなく、初動方向自体が誤っている可能性が高まった。

24.2.14 20260630_165714

サンプル 0、summary なし。実験記録ではバッテリー 0% 表示による飛行前中断相当として扱われる。制御性能データではなく、飛行前安全判定の記録である。

24.2.15 20260630_165807

x 誤差 659mm。前回より逆走量は減ったが、方向問題は残った。z 誤差 11mm は良好で、この時点では主問題が x に集中していた。

24.2.16 20260630_170026

100mm 級の短距離試験。x 誤差 45mm、y1mm、stale 81%。途中までは目標方向へ進んだため、一時的に「符号ではなく stale 判定が厳しすぎる」と解釈された。後のログで撤回された仮説である。

24.2.17 20260630_170651

stale は 3.7% と低いのに x 誤差 2415mm、x 速度ピーク 951mm/s。センサ更新停止では説明できず、固定 `pitch_sign=-1` が実機に対して逆であることを決定づけた。前ログの解釈を反証した。

24.2.18 20260630_171035

逆走は止まったが、z 誤差 27mm、y 振動 3 回。目標高度近傍で throttle の正負反転が観測され、高度軸の最小出力とゲインが強すぎる問題へ焦点が移った。

24.2.19 20260630_172047

x27mm、y-8mm と改善したが z52mm。x 速度ピーク 509mm/s で、目標付近の制動と高度保持の両立が未完成。

24.2.20 20260630_172711

stale73%、x 誤差 1066mm。位置更新の粗さが再び支配的になり、stale 中に即ブレーキする方式が早すぎる停止を生むことが分かった。

24.2.21 20260630_174211

18.13 秒で runaway。最終誤差だけなら x73mm、y36mm、z0.6mm だが、最良到達点から再び離れ続けたため安全停止した。最終瞬間の値だけでなく、途中経過を終了理由へ使う思想を示す。

24.2.22 20260707_164639

4.27 秒で runaway。x/y は数 cm だが z70mm。速度ピーク x346mm/s。多軸高速化直後で、初動の強さに対して runaway ガードが早期作動した。

24.2.23 20260707_164950

2.07 秒、x 誤差 302mm、x 速度 653mm/s。主軸 x が高速で悪化し、ガードが停止。最大加速と方向・制動条件がまだ整合していない。

24.2.24 20260707_165019

1.67 秒、y 誤差 233mm、y 速度 367mm/s。x より横軸が支配的に崩れた。多軸化で roll 側の符号・座標・ゲインも独立検証が必要になった。

24.2.25 20260707_165327

0.99 秒、x 誤差 168mm、x 速度 395mm/s。極短時間で停止しており、runaway ガードが安全側に敏感すぎる可能性と、初動方向の問題の両方がある。

24.2.26 20260707_165345

0.66 秒、y 誤差 527mm、y 速度 624mm/s。わずか 9 サンプルで横方向が大きく悪化。y 軸の不安定性が最も明確な runaway 例の一つ。

24.2.27 20260707_165911

1.73 秒、x 誤差 185mm、x 速度 382mm/s。x 軸高速初動で停止。ガードは事故回避には有効だが、目標到達には至っていない。

24.2.28 20260707_165932

1.84 秒、x159mm、y117mm、y 速度 496mm/s。複数軸が同時に悪化し、単軸別チューニングだけでは足りない。

24.2.29 20260707_170225

1.91 秒、x224mm、z83mm。水平だけでなく高度も同時に外れた。高速水平移動時の鉛直推力不足を疑う根拠になる。

24.2.30 20260707_170244

3.15 秒、y350mm、z85mm、z 速度 231mm/s。横流れと高度変動の組合せ。水平傾斜時の throttle 補償へつながる種類の失敗。

24.2.31 20260707_170506

2.13 秒、x406mm。y は-8mm と良い。主軸だけが離脱しており、軸間問題より x の高速フェーズ条件が支配的。

24.2.32 20260707_170526

2.47 秒、y383mm、z126mm。x は-45mm。y 高速移動と高度維持の両方が失敗。

24.2.33 20260707_170935

runaway 即停止を避け、19.95 秒制御継続できたが timeout。x-34mm、y61mm、z63mm。安全停止過敏性は改善した一方、整定速度が不足。

24.2.34 20260707_171019

約 20 秒 timeout。x-17mm は良いが、y47mm、z46mm。x 単独なら近いが、多軸完成条件には未達。

24.2.35 20260707_171632

timeout を 35 秒へ延長しても x-58mm、y-61mm、z62mm。振動 x9、y11。時間を延ばすだけでは収束せず、制御則の変更が必要。

24.2.36 20260707_171724

35 秒 timeout。y 速度ピーク 472mm/s、x/y 振動。横軸の高速度・制動不足が目立つ。

24.2.37 20260707_172317

z 誤差 798mm、z 速度 741mm/s。高度が重大に崩れた。高度回復ガードと altitude fault の必要性を強く示す。

24.2.38 20260707_172418

x-4.65mm と非常に良いが y69mm、z30mm。再び「主軸成功だけでは全体成功でない」例。

24.2.39 20260707_173145

x/y それぞれ 26 回の振動。最終誤差は x18mm、y-81mm、z-20mm だが、途中で大きな往復があった。ドリフト・振動ガードの対象。

24.2.40 20260707_173239

y 最終誤差-1.17mm、z7.53mm は良いが x128mm。振動は x12、y25、z30。最終値だけなら 2 軸成功に見えるが、経路は非常に不安定。

24.2.41 20260707_173920

1.08 秒で `attitude_fault`。yaw/姿勢が 55 度超へ崩れたため、水平位置より方位安全を優先して停止した。新しい安全ガードが実機で作動した証拠。

24.2.42 20260707_173949

35 秒 timeout。x109mm、y-83mm、z-3mm。z 最終値は良いが、y/z 振動 27 回。高度の最終一致は安定した飛行を意味しない。

CHAPTER 25

24. 全 25 コミットの変更量と、その意味

日時	hash	件名	変更量	意味
05-19 16:41	`90ddb8b`	initial	7 files, +103	PyCharm プロジェクト開始
05-19 17:05	`1378ff4`	チャイム、離着陸	3 files, +86/-18	初実機通信・基本動作
05-19 17:15	`c10d4d8`	ホバリング	2 files, +27/-5	安定待ちの導入
05-19 18:24	`4ba456e`	前後左右・yaw・上下	4 files, +79/-7	基本運動軸の確認
06-02 16:23	`09a1ad5`	課題 4.1	20 files, +378/-40	センサ・周辺 API の分離試験
06-02 16:29	`b60bb33`	課題 4.2	1 file, +14	ユーザ入力による回転
06-23 16:23	`9114c3c`	PID でいろいろ	7 files, +777/-99	開ループから PID へ
06-23 17:02	`d697110`	共通制御基盤	3 files, +660/-406	重複排除、ログ、自動調整
06-23 17:51	`76274be`	場面別ゲイン	1 file, +420/-94	軸・状況別の非対称化
07-07 17:12:07	`75df208`	実験仕様・記録	6 files, +1795	実験知識を文書化
07-07 17:12:13	`c13d663`	多軸 PID	3 files, +1360/-554	x/y/z/yaw 同時制御
07-07 17:12:19	`c144e34`	ブレーキ確認	1 file, +11	着陸・停止の小試作
07-07 17:12:27	`bf0d1ff`	実機ログ・プロファイル	41 files, +11706	実験データを成果物化
07-07 17:12:36	`700c91f`	workspace	1 file, +60/-56	IDE 状態更新
07-07 17:12:58	`b01dfef`	回帰テスト	1 file, +278	実機バグをテストへ固定
07-07 17:13:36	`a506470`	timeout・ゲイン	1 file, +33/-21	実機周期へ適応
07-07 17:47	`c001264`	追加ログ	7 files, +2924	runaway 群の保存
07-07 18:21	`f6232eb`	PID 限界・仕様再設計	4 files, +443	ハイブリッド制御へ思想転換
07-07 18:29:58	`2314db9`	制御・テスト更新	2 files, +499/-38	座標変換・速度サーボ・安全強化
07-07 18:30:03	`2f57132`	実験記録追記	1 file, +330	結果と仮説を同期
07-07 18:30:09	`a7e88bb`	ログ・解析更新	2 files, +3017/-5	変更直後結果を保存
07-07 18:38	`10fc6f3`	漏洩経路遮断	56 files, +68/-17916	IDE・ログ・秘密情報を除外
07-14 18:26	`693f019`	加速度位置推定	1 file, +257	optical flow 代替案
07-14 18:49	`996a5c3`	姿勢制御堅牢化	1 file, +251/-66	yaw、ZUPT、座標変換強化
07-21 17:09	`011b1d1`	カラーセンサ	1 file, +10/-3	2cm 制約・3 秒焼き付き発見

変更量の意味を読む際には注意が必要である。行数が多いコミットほど優れているわけではない。たとえば `bf0d1ff` の 11,706 行は主に実機ログであり、`f6232eb` の 443 行は制御思想を変えた仕様書である。また `10fc6f3` の 17,916

行削除は機能削除ではなく、生成ログを Git 追跡から外した結果である。

CHAPTER 26

25. 全 35 テストを一件ずつ説明する

26.1 25.1 加速度位置制御 12 件

1. `test_calibration_uses_mean_raw_acceleration` 校正が最後の一値や中央値ではなく、収集した raw 値の平均をバイアスとして使うことを確認する。静止ノイズを平均化する目的である。

1. `test_cli_default_inverts_codrone_x_acceleration_seen_in_logs` 実機ログで観測した x 加速度の向きに合わせ、CLI 既定符号が反転になっていることを固定する。符号退行による逆走を防ぐ。

1. `test_command_signs_can_be_inverted_for_field_validation` 現場で機体個体・API 解釈が違った場合、推定符号と指令符号を設定で反転できることを確認する。

1. `test_output_is_limited_by_configured_power` 大誤差でも pitch/roll が安全上限を越えないことを確認する。数式が正しくても出力飽和処理がなければ危険である。

1. `test_positive_x_target_commands_positive_pitch` 正 x 目標が正 pitch を作るという最も基本的な方向対応を固定する。

1. `test_positive_y_target_commands_left_roll_negative` このコードの +y を左と定義し、実機 API では左移動が負 roll であることを固定する。

1. `test_acceleration_axis_sign_can_be_inverted` raw センサ軸の符号補正が推定加速度へ正しく反映されることを確認する。

1. `test_bias_corrected_constant_acceleration_integrates_position` バイアス除去後の一定加速度が速度・位置へ積分される基本数式を検証する。

1. `test_deadband_suppresses_small_bias_noise` 小さい残留ノイズが速度・位置へ無限累積しないよう 0 へ落とすことを確認する。

1. `test_raw_acceleration_unit_conversion_matches_codrone_documented_scale` raw 単位から物理加速度単位への換算が想定どおりであることを固定する。

1. `test_flight_and_hovering_state_is_success_even_when_height_sensor_is_zero` 高度センサが 0 でも、flight state と movement state が飛行中でエラーがなければ離陸成功とする。壊れた高度一値だけで誤停止しないためである。

1. `test_state_error_keeps_takeoff_check_failed` 飛行状態らしく見えても state error があれば成功扱

いしない。複数情報のうち安全エラーを優先する。

26.2 25.2 drone_precision 17 件

1. test_yaw_command_sign_matches_observed_codrone_direction 実機で確認した yaw 符号-1 を固定し、プロファイル学習で変わらないようにする。

1. test_body_to_world_inverts_world_to_body 座標変換を往復したとき元の値へ戻ることを確認する。変換式の符号・sin/cos 配置ミスを防ぐ。

1. test_world_to_body_keeps_axes_when_yaw_is_zero yaw 0 では世界 x/y と機体前後/左右が一致するという基準例。

1. test_world_to_body_rotates_error_by_current_yaw yaw 90 度のとき世界前方誤差が機体横方向になることを確認する。

1. test_horizontal_drift_guard_limits_growing_oscillation 誤差符号が反転しながら振幅増大する系列で、ガード scale が 1 未満になることを確認する。

1. test_integral_does_not_grow_while_output_is_saturated 出力上限へ張り付いている間に I 項だけが蓄積し、後で大オーバーシュートを起こさないようにする。

1. test_minimum_output_is_disabled_near_target 小誤差でも強制最小出力を出し続ける振動を防ぐ。

1. test_lateral_move_uses_full_roll_acceleration_and_braking 長距離 y 移動で roll 最大加速から最大制動へ遷移することを確認する。

1. test_long_move_uses_full_acceleration_and_full_braking 長距離 x 移動で pitch 最大加速と最大制動を使うことを確認する。

1. test_vertical_move_uses_throttle_rapid_phase z 長距離でも高速フェーズが適用されることを確認する。

1. test_close_fast_motion_is_braked 目標に近く、目標方向速度が高い場合、通常 PID が前進を要求しても逆向きブレーキになることを確認する。

1. test_premature_reverse_command_is_forced_back_toward_target D 項や速度項が遠方で早すぎる逆指令を作った場合、最小駆動力で目標方向へ戻す。

1. test_velocity_servo_brakes_when_close_and_fast 近傍速度サーボが到達可能速度を越えた進入を検知して制動する。

1. test_velocity_servo_drives_back_when_far_and_slow 遠方で低速なら、符号を含め正しい目標方向

へ十分な出力を作る。

1. `test_velocity_uses_time_between_fresh_position_packets` 同じ位置キャッシュを何回読んだかではなく、新しい位置パケット間の実時間で速度を計算する。

1. `test_includes_physical_braking_latency_and_margin` 停止距離が純粋な $v^2/2a$ だけでなく、センサ遅延中の進行距離と余裕を含む。

1. `test_stationary_distance_is_margin_only` 速度 0 なら運動制動距離は 0 で、残るのは安全余裕だけであることを確認する。

26.3 25.3 時間較正 6 件

1. `test_plan_move_splits_long_segments` 長時間連続指令を最大 2 秒以下の区間へ分割し、途中で停止・補正機会を作る。

1. `test_plan_move_uses_direction_specific_signs` forward/back/left/right で別々の正負指令を使う。

1. `test_update_direction_uses_measured_speed` 人間が測った距離を実行時間で割り、方向別速度較正を更新する。

1. `test_update_landing_keeps_takeoff_trim` 着陸時の補正を変えても通常飛行 trim が失われない。

1. `test_update_landing_rejects_positive_throttle` 手動下降設定へ正の throttle を入れて上昇させる設定ミスを拒否する。

1. `test_update_trim_keeps_motion_calibration` trim 変更時に前後左右の速度較正を消さない。

26.4 25.4 テスト結果の正しい解釈

35 件全成功という事実は、「実装された数式と安全条件が、用意した入力例に対して期待どおり」という意味である。以下は意味しない。

- センサ換算値が実機で常に正しい。
- fake drone が実機の内部制御を完全再現する。
- 風がある部屋で目標へ到達する。
- 実際の制動加速度が設定値 $90\mathrm{mm/s^2}$ である。
- 30cm 移動の誤差が許容内になる。
- カラーセンサのフリーズが直った。
- 高度回復指令が必ず墜落を防ぐ。

テストは必要条件であり、実機成功の十分条件ではない。

CHAPTER 27

26. 用語と数式を省略せず説明する

27.1 26.1 PID

PID は、目標と現在値の差を `error` として、次の三成分を組み合わせる。

- P（比例）: 今の誤差が大きいほど強く動かす。
- I（積分）: 過去から残り続ける誤差を蓄積し、風などの定常外乱へ勝つ。
- D（微分）: 誤差の変化速度を見て、目標へ高速接近中なら出力を弱める。

このプロジェクトではさらに `kv × measurement_rate` を引き、実測速度へ直接減衰をかける。D と速度減衰は似ているが、D は誤差変化、kv は状態速度を明示的に使う。

27.2 26.2 アンチワインドアップ

モータ指令は最大 100 である。計算上 200 が必要でも実機は 100 しか出せない。この間 I 項を蓄積すると、誤差が小さくなった後も大きな積分が残り、逆側へ飛び越える。出力飽和中は積分を増やさない、または積分を上限へ制限する処理がアンチワインドアップである。

27.3 26.3 stale

制御ループが新しい位置値を得られず、前と同じ位置パケットを読んでいる状態。機体が本当に静止して同じ位置なのか、センサ更新が止まったのかは値だけでは区別しにくい。そのためセンサ側時刻、位置変化、wall time を組み合わせる。

27.4 26.4 runaway

単に誤差が大きい状態ではない。一度近づいた最良点から、複数の新鮮な更新にわたって再び離れ続ける状態を指す。目標が遠い初期状態を runaway と誤認しないよう、最良誤差と初期誤差を基準にする。

27.5 26.5 ZUPT

Zero Velocity Update。加速度、gyro、指令がすべて小さく「停止している可能性が高い」と判定したとき、積分速度を明示的に 0 へ戻す。IMU 積分のドリフトを止めるが、移動中に誤判定すると実速度を消す。

27.6 26.6 dead reckoning

外部位置を直接見ず、既知の速度モデルと経過時間から移動量を積み上げる推測航法。コマンドどおり動いた前提なので、風で押された移動を観測できない。

27.7 26.7 世界座標と機体座標

- 世界座標: 離陸時の前方を x、左を y と固定する座標。
- 機体座標: 現在の機首前方と左右を基準にする座標。

yaw が変わると二つは回転してずれる。世界 x へ進みたいとき、機首が 90 度回っていれば pitch ではなく roll 相当の動きが必要になる。座標変換なしの制御が円運動し得る理由である。

27.8 26.8 停止距離

現在速度 v、制動加速度 a なら、理想的な停止距離は $v^2/(2a)$。しかしセンサ値を読み、Python が計算し、次指令が届くまで遅延する。その間にも `v × latency` 進む。さらにモデル誤差用 margin を加える。

27.9 26.9 timeout

指定時間内に settled にならなかった終了。必ずしも暴走ではない。安全に飛び続けたが誤差・速度条件を満たせなかった場合も timeout である。したがって timeout ログは、runaway より穏やかでも成功ではない。

27.10 26.10 settled

一瞬位置許容内へ入るだけではない。位置誤差と速度が許容内に入り、その状態が一定時間連続した場合の完了。勢いよく通過しただけのオーバーシュートを成功にしない。

CHAPTER 28

27. 事実・推定・解釈の証拠台帳

28.1　27.1 直接確認できる事実

- Git コミット日時、hash、変更量。
- 現在のコードと設定値。
- JSONL/CSV/summary に保存された数値。
- 35 テストが成功したこと。
- 利用者が述べた再現性、上下非対称、カラー 2cm、3 秒焼き付き、optical flow 評価。
- コミットメッセージに「PID 制御に限界を感じた」とあること。

28.2　27.2 強い推定

- 低 stale 逆走ログの直後に符号固定と runaway が入ったため、逆走が変更動機。
- 高度大崩れログと高度回復ガードの対応。
- raw 加速度 0 ログの後に姿勢角由来加速度が追加されたため、量子化回避が動機。
- 大量の重複コード削除と共通入口化から、保守性・修正一貫性が動機。

28.3　27.3 断定できない解釈

- 開発者がその瞬間に感じた感情。
- 個々の runaway が風、センサ、符号、姿勢のどれ一つだけで起きたか。
- カラー焼き付きがハードウェア、ファームウェア、API キャッシュのどこで起きたか。
- optical flow stale の内部原因。
- 加速度換算定数が全個体・全姿勢で正しいか。

本報告書で思想を述べる場合、コミット順と文書に繰り返し現れる優先順位から復元している。内面そのものを Git が記録しているわけではない。

CHAPTER 29

28. 完全版としての最終結論

この研究で最も大きかった変更は、PID ゲインの数値ではない。問題の捉え方が変わったことである。

初期には、機体へ命令を出せば、その命令に対応する動きが得られると考えられる段階だった。前進、後退、左右、上下、yaw、図形飛行、時間飛行、手動操作を行い、API と機体の対応を学んだ。しかし実機では、同じ時間・同じ出力でも同じ距離にならなかった。そこで位置を読み、誤差が残る限り戻す PID へ進んだ。

PID を実装すると、次は制御器ではなく観測と座標の問題が表面化した。古い高度 0 を基準にすれば、正しい PID でも急下降する。pitch 符号が逆なら、ゲインをどれだけ調整しても逆走する。yaw がずれれば、世界座標の x 誤差を pitch へ入れても方向が合わない。位置パケットが止まれば、D 項と速度推定が壊れる。上昇と下降の物理応答が違えば、一つの対称ゲインは振動を作る。

それらを一つずつ切り分けるため、試験距離を 2m から 300mm、100mm へ下げた。stale 回数を実時間へ変えた。一回のログから作った仮説を、次の反証ログで撤回した。異常ログを自動学習へ使わないようにした。x だけの最終誤差ではなく、y、z、yaw、速度、振動、センサ鮮度、終了理由を保存した。

その結果、「PID をさらに強くする」方向から、状態機械、速度サーボ、停止距離、yaw 優先、高度回復、runaway 停止というハイブリッド方式へ進んだ。速くするために最大加速を使い、速く止まるために最大制動を使い、正確に止まるために近傍だけ低速化した。安全性は速さの反対ではなく、最大性能を試すための条件として扱われた。

しかし、最終的に最も大きな限界は optical flow による位置観測だった。そこで加速度二重積分、姿勢角由来加速度、ZUPT、時間較正、コマンド積算、壁 ToF 補正、色マーカー補正を考えた。各方式には明確な長所と限界がある。加速度は短時間変化を捉えるがドリフトする。時間較正は単純だが風を見ない。コマンド積算は滑らかだがモデル外の移動を見ない。壁 ToF は壁がある時だけ使える。カラーは約 2cm まで近づく必要があり、約 3 秒焼き付く。

したがって、現時点で科学的に正しい結論は次である。

1. 内蔵センサが健全な条件では、相対移動精度を改善する制御は構築できる。
2. どの単一センサも、すべての飛行条件で真位置を保証しない。
3. センサが真位置を失った後、観測なしに厳密な絶対位置を復元することはできない。
4. 最速・精密・堅牢を目指すなら、複数情報を有効範囲ごとに使い分け、不確実性が限界を越えたら停止する必要がある。
5. 現在の 35 テスト成功はソフトウェアの重要な進歩だが、実機完成の証明ではない。
6. 次の研究中心はゲインの微調整ではなく、センサ故障モードの定量化、外部真値による評価、短時間 sensor fusion、方向別・軸別の物理モデル化である。

以上が、Git、コード、全飛行ログ、加速度ログ、dead reckoning ログ、球風ログ、仕様書、テスト、および利用者から提示された実機観察を統合して得られる、現時点の完全な結論である。

CHAPTER 30

29. PID 以外の現存実験ログを一件ずつ確認した台帳

30.1 29.1 球風実験 7 件

30.1.1 ball_wind_20260714_173950.jsonl

- レコードは metadata 1 件だけ。
- 実験設定の保存までは行われたが、preflight、離陸、風パルス、summary はない。
- 初期起動直後に中断した、または処理が実験本体へ進まなかったログと考えられる。
- 実測性能評価には使えない。

30.1.2 ball_wind_20260714_174007.jsonl

- 全 19 レコード。
- metadata、preflight、before_blow、sample 15、error 1。
- エラーは「球が遠すぎます: 999.9cm。18.0cm 以下にしてください」。
- 999.9cm は実距離 999.9cm を高精度に測ったというより、前方対象を検出できない場合の上限・無効値と解釈するのが自然。
- この失敗により、離陸後の front range で床上 pickleball を監視する方式が不適切と分かった。
- 後の設計では球の距離確認を離陸前だけに限定し、離陸後の front range は参考値へ格下げした。

30.1.3 ball_wind_20260714_175404.jsonl

- 全 4 レコード。
- metadata、preflight、sample 1、error 1。
- エラーは「高度が低すぎます: 0.0cm。25.0cm 以上を安全条件にしています」。
- 実際に高度 0 で飛んでいたと断定できない。高度センサ未更新、離陸状態不成立、センサ 0 値の可能性がある。
- 安全ロジックは接触リスクを避けるため中止した。
- これは安全側として正しいが、高度 0 の意味を別途離陸状態と組み合わせる必要性を示し、離陸診断コードの背景と整合する。

30.1.4 ball_wind_20260714_175438.jsonl

- 全 33 レコード。
- metadata、preflight、before_blow、sample 27、wind burst start/end、summary が揃う。
- pitch 10、0.5 秒、1 パルスの試験。
- 実測球移動量は未入力。
- 飛行手順が最後まで通ったことは確認できるが、30cm 目標の成否は判定不能。

30.1.5 ball_wind_20260714_175507.jsonl

- 全 33 レコードで構造は 175438 と同じ。
- 同条件反復に相当する。
- 実測値未入力のため、再現性の数値比較はできない。

30.1.6 ball_wind_20260714_175824.jsonl

- 全 33 レコード。
- 同じく風パルスと summary まで完了。
- 実測値未入力。
- 3 回同条件で試す実験思想に沿うログ群だが、目的変数が欠けている。

30.1.7 ball_wind_20260714_180329.jsonl

- 全 33 レコード。
- 後期設定では `preflight_ball_check_enabled`、前方距離最小・最大が明示され、設定名称が整理された。
- 実測値未入力。
- コード改善後の手順完走は確認できるが、球移動性能は未評価。

球風ログ全体の結論

安全手順とログ機構は段階的に改善した。最初は離陸後 front range を球の安全距離に使おうとして 999.9cm で失敗し、次に高度 0 で安全停止し、その後 4 回は風パルスまで完走した。しかし最重要の `actual_ball_move_cm` が全て欠けるため、制御変数と球移動量の関係は一切確定できない。

30.2 29.2 初期加速度位置 CSV 10 件

30.2.1 accel_position_20260714_174347.csv

- 65 サンプル。
- 30cm 目標。
- 最終推定 x`-15.68cm`、y`4.43cm`。
- x 誤差 `45.68cm`。
- 推定方向が目標と逆。符号、バイアス、加速度積分のいずれかが不整合。

30.2.2 accel_position_20260714_174436.csv

- 43 サンプル。
- 最終推定 x/y とも 0。
- 30cm 目標に対し誤差 30cm。
- raw 水平加速度が deadband 内または 0 に量子化され、推定器が移動を認識しなかった。

30.2.3 accel_position_20260714_174455.csv

- 64 サンプル。
- 推定 x/y とも 0、x 誤差 30cm。
- 前ログと同じゼロ張り付きが再現。偶発的一回ではない。

30.2.4 accel_position_20260714_174527.csv

- 129 サンプル。
- 目標を 60cm へ伸ばしても推定 x/y は 0。
- 実験時間や目標を増やしても raw 水平信号が観測されなければ推定は動かない。

30.2.5 accel_position_20260714_174918.csv

- 53 サンプル。
- 30cm 目標、推定 0。
- 出力を変えた後も位置推定は反応しておらず、制御ゲインより観測側が一次問題。

30.2.6 accel_position_20260714_175055.csv

- ヘッダ相当の 1 行だけで、有効サンプルなし。
- 起動直後中断またはログ初期化のみ。

30.2.7 accel_position_20260714_175224.csv

- 有効サンプルなし。
- 制御性能評価対象外。

30.2.8 accel_position_20260714_175559.csv

- 有効サンプルなし。
- 制御性能評価対象外。

30.2.9 accel_position_20260714_175852.csv

- 49 サンプル。
- 30cm 目標。
- 最終推定 x-150.29cm、y9.73cm。
- x 誤差 180.29cm。
- ゼロ張り付きとは逆に、残留加速度が二重積分され大発散した。
- 同じ推定器が「全く動かない」と「大きく動きすぎる」の両極端を示し、deadband・バイアス設定の難しさを示す。

30.2.10 accel_position_20260714_180130.csv

- 有効サンプルなし。
- 制御性能評価対象外。

初期加速度ログ全体の結論

有効データ 6 件のうち、4 件は推定 0、2 件は逆方向ドリフト。summary がある 4 件はすべて timeout。raw 加速度単独の長時間二重積分は成立していない。

30.3 29.3 改良 inertial XY ログ 8 件

30.3.1 inertial_xy_20260721_162442.csv

- 160 サンプル。
- 30cm 目標。
- 最終推定 x5.19cm、y35.40cm。
- x 誤差 24.81cm、y 誤差-35.40cm。
- 主方向へほとんど進まず、横方向推定が大きく増えた。軸符号、姿勢、roll 指令、加速度変換の混在が疑われる。

30.3.2 inertial_xy_20260721_162534.csv

- 36 サンプル。
- 推定 x22.43cm、y-0.18cm。
- x 誤差 7.57cm。
- ログ上は比較的目標へ近い短時間例。ただし外部真値がないため、推定が正しいかは不明。

30.3.3 inertial_xy_20260721_162638.csv

- 46 サンプル。
- 推定 x22.88cm、y1.12cm。
- x 誤差 7.12cm。
- 前ログと似た内部推定結果で、短時間の再現性は見える。

30.3.4 inertial_xy_20260721_162737.csv

- 106 サンプル。
- 推定 x45.75cm、y-24.76cm。
- x は目標を 15.75cm 越え、横にも 24.76cm ずれた。
- 長時間化すると積分誤差が増える傾向と整合する。

30.3.5 inertial_xy_20260721_170652.csv

- 81 サンプル。
- IMU、姿勢由来、使用加速度を分けて記録する新形式。
- 推定 x8.38cm、x 誤差 21.62cm。
- 姿勢由来加速度を導入しても十分進んだと認識できなかった。

30.3.6 inertial_xy_20260721_171307.csv

- 161 サンプル。
- 推定 x/y とも 0、x 誤差 30cm。
- 目標速度は出続けるが採用加速度が 0 になり、推定位置が変わらない。
- 姿勢角の粗さまたは機体内部姿勢保持により、`sin(angle difference)` が移動を表現しなかった可能性。

30.3.7 inertial_xy_20260721_171423.csv

- 267 サンプル。
- 推定 x14.73cm、y-40.07cm。
- 最終 yaw 表示 160 度。
- 大きな yaw ずれに対して機体/世界座標変換と yaw 保持が追いつかず、横推定が崩れた。

30.3.8 inertial_xy_20260721_171507.csv

- 265 サンプル。
- 推定 x/y とも 0、x 誤差 30cm。
- 最終 yaw 差は約 4.14 度で極端ではないが位置 0。
- yaw だけが原因ではなく、加速度観測自体が不足している。

改良 inertial ログ全体の結論

短時間 2 件では内部推定上約 22〜23cm まで進んだが、別条件ではゼロ張り付き、横ドリフト、オーバーシュート、大 yaw ずれが起きた。IMU・姿勢・blend のいずれも、外部真値なしに信頼できる長時間位置へはなっていない。

30.4 29.4 command dead reckoning ログ 3 件

30.4.1 command_dead_reckoning_20260721_172153.csv

- 70 サンプル。
- `default_mission` 系の方向動作ログ。
- 最終 step は `right`。
- 推定 x-1.21cm、y-0.26cm、高度 79.3cm、色 `light blue`。
- 複数方向移動後、モデル上はほぼ原点へ戻った。
- これはコマンドモデルの閉路整合性を示すが、実機が本当に原点へ戻った証明ではない。

30.4.2 command_dead_reckoning_20260721_172837.csv

- 90 サンプル。
- 80cm 目標相当。
- 推定 x75.15cm、y0.44cm、高度 73.7cm。
- 最終誤差 4.87cm。
- 直近平均 4.92cm、標準偏差 0.16cm、95% 相当上側 4.98cm。
- 統計停止条件に対し十分安定。

30.4.3 command_dead_reckoning_20260721_172907.csv

- 90 サンプル。
- 推定 x75.16cm、y-0.39cm、高度 78.1cm。
- 最終誤差 4.85cm。
- 平均 4.90cm、標準偏差 0.16cm、上側 4.97cm。
- 直前ログとほぼ同一の内部推定結果。

dead reckoning ログ全体の結論

内部モデルの再現性は高い。特に 2 回の 80cm 試験がほぼ同じ推定値で停止した。しかし推定値は「出した指令をモデルへ積分した結果」なので、実機が風で押された、推力が弱かった、滑ったという誤差は観測しない。成功判定には外部実測が必要である。

CHAPTER 31

30. 作業ツリーに残る未コミット状態の意味

報告書拡張時点で、`master` は `origin/master` より 1 コミット先行している。先行コミットはカラーセンサ実験である。

追跡済み変更:

- `experiment_logs/README.md`: 球風実験文書への索引追加。
- `simple_accel_xy.py`: raw IMU 中心から、姿勢由来加速度、blend、速度サーボ、yaw 保持へ大幅変更。

未追跡:

- `accel_position_control.py`
- `ball_wind_experiment.py`
- `codrone_takeoff_diagnose.py`
- `command_dead_reckoning.py`
- `time_calibrated_xy.py`
- `test_accel_position_control.py`
- `test_time_calibrated_xy.py`
- 加速度・球風ログ
- 球風実験文書
- 本報告書

この状態は、これらの方式がまだ正式な一つの制御方針へ統合されていないことを示す。未追跡だから価値がないわけではない。むしろ、optical flow の限界を受けて複数代替案を急速に比較している研究途中の状態である。ただし、Git から見れば変更理由・対応ログ・テストが一つのコミットへ固定されておらず、後から因果関係を追いにくい。

今後コミットする場合は、少なくとも次を分けるべきである。

1. 加速度推定本体とテスト。
2. 時間較正方式とテスト。
3. command dead reckoning と実験ログ要約。
4. 離陸診断。
5. 球風実験ツールと実験記録。
6. 生ログは `.gitignore` 方針に従い、要約だけを Git へ残す。

これは見た目の履歴整理ではなく、「どの仮説を、どのコードで、どのデータを使って検証したか」を保存するためである。

Evidence before assumption.
Safety before speed.
Observability before control.

CoDrone EDU Control Research / PythonProject